

Leveraging Analytics to Combat Fraud in Self-Checkout Systems

Developing a Predictive Model to Enhance Security and Customer Trust in Retail Self-Scanning Processes

Analytics Project 2025

Author: Jibin Mathew Peechatt, Leon Bytyqi, Jevoen Jenifar,
and Sascha Vetsch

Lecturer: Prof. Andreas Reber

City, Date: Brugg-Windisch, 24.05.2025

Analytics Project (BA) 2025

Authors

Jibin Mathew Peechatt, jibinmathew.peechatt@students.fhnw.ch

Sascha Vetsch, sascha.vetsch@students.fhnw.ch

Leon Bytyqi, leon.bytyqi@students.fhnw.ch

Jevoen Jenifar, jevoen.jenifar@students.fhnw.ch

Lecturer

Prof. Andreas Reber

University of Applied Sciences and Arts

Northwestern Switzerland

andreas.reber@fhnw.ch

Brugg, May 2025

Declaration of Authenticity

We hereby declare

- that we have written this performance record independently and on our own;
- that we have correctly cited all text passages not originating from ourselves according to common scientific citation rules and that we have clearly mentioned the sources used;
- that we have declared in an index all aids used (e.g. AI tools such as chatbots [e.g. ChatGPT], translation/paraphrasing tools) or programming applications [e.g. Github Copilot] and have indicated their use at the corresponding text passages;
- that we have acquired all intangible rights to any materials we may have used, such as images or graphics, or that these materials were created by us;
- that the topic, the paper or parts of it have not been used in a performance record of another module, unless this has been expressly agreed with the lecturer in advance and is indicated in the paper;
- that we are aware that our work may be checked for plagiarism and for third-party authorship of human or technical origin (artificial intelligence);
- that we are aware that the School of Business FHNW will pursue a violation of this declaration of authenticity or its underlying obligations of the study and examination regulations of the School of Business FHNW and that disciplinary consequences (reprimand or expulsion from the study program) may result.

Brugg, 24th May 2025

Signatures

Jibin Mathew Peechatt



Leon Bytyqi



Sascha Vetsch



Jevoen Jenifar

Jenifar
Jevoen



Digital signiert von Jenifar Jevoen
DN: cn=Jenifar Jevoen,
email=Jevoen.Jenifar@zuerich.ch
Datum: 2025.01.03 08:33:55
+01'00'

Management Summary / Abstract

The Analytics Project 2025 focuses on a well-established food retailer that has introduced self-scanning systems in its stores to speed up the checkout process and make it more convenient for customers. While this innovation offers operational benefits, it also introduces the risk of fraud; studies show that around 5% of transactions may include unscanned items. The retailer's objective was to minimise these losses without disrupting the shopping experience of honest customers. Therefore, the project aimed to develop a classification model that could accurately detect fraudulent transactions with minimal false positives. The team adopted the CRISP-DM framework to structure the process, guiding the project through six phases: business understanding, data understanding, data preparation, modelling, evaluation and deployment planning.

During the analysis of the data, it became clear that fraudulent activity had occurred almost exclusively among customers with trust levels 1 and 2. This insight enabled a more focused dataset to be created, thereby improving model efficiency. Additional features, such as 'Total Items' and 'Average Value Per Item', were developed to improve predictive accuracy. Various machine learning models were evaluated, including Random Forest, Logistic Regression, SVM and XGBoost. Two modelling strategies were employed: one focused on balanced performance through standard cross-validation and the other incorporated cost-sensitive learning to prioritise reducing expensive false positives. The performance of each model was evaluated using metrics such as the ROC AUC, the F1 score, precision and recall.

The evaluation phase confirmed that the XGBoost model using the second strategy (cost-sensitive learning with threshold adjustment) offered the best results. With an F1 score of 0.9135, a precision of 0.8701 and a recall of 0.9614, it surpassed the project's success criteria of achieving a minimum precision and recall of 0.80. These outcomes demonstrate that the final model can effectively identify fraudulent behaviour while minimising disruption for legitimate customers. Consequently, the XGBoost classifier from strategy 2 was chosen for implementation, allowing the retailer to maintain a secure and efficient self-checkout system that preserves customer confidence and minimises financial losses.

Table of Contents

Management Summary / Abstract.....	III
Table of Contents.....	IV
1 Introduction	1
2 Business Understanding	2
2.1 Background Information.....	2
2.2 Business Objective, Risks and Benefits	3
2.3 Success Criteria and Data Mining Goals	3
2.4 Project Phases and Tool.....	4
3 Data Understanding	6
3.1 Description of the Data	6
3.2 Further Exploration of the Data.....	7
3.2.1 Exploration in Jupyter Notebook.....	7
3.2.2 Exploration in Orange.....	15
3.3 Verification of the Data Quality	17
4 Data Preparation.....	18
4.1 Missing Values (NA) and Duplicates	18
4.2 Construct Data.....	18
4.3 Balancing Data Set.....	20
4.4 Reducing categories / grouping	20
4.5 Feature Selection.....	21
4.5.1 Random Forest Classifier	22
4.5.2 XGBoost Classifier.....	22
4.5.3 Logistic Regression	23
4.5.4 SVM.....	23
4.6 Formatting the Data	24
5 Modeling	26

5.1	XGBoost Classifier.....	26
5.1.1	Understanding the XGBoost Classifier	26
5.1.2	Why is the XGBClassifier suitable for our case?	26
5.1.3	How Can We Use It in Our Project?	27
5.2	Logistic Regression	27
5.2.1	Understanding Logistic Regression.....	27
5.2.2	Why Is Logistic Regression a Good Selection for This Case?	27
5.2.3	How We Can Use It in Our Project	28
5.3	Support Vector Machines	28
5.3.1	Understanding SVM	28
5.3.2	Why Is SVM a Good Selection for This Case?	28
5.3.3	How We Can Use It in Our Project	29
5.4	Random Forest Classifier	29
5.4.1	What is a RandomForestClassifier?	29
5.4.2	Why is this model good for finding fraud at self-checkout?	30
5.4.3	How can we use the RandomForestClassifier in our project?	30
5.5	Generation of Test Design.....	30
5.6	Building the Model	31
5.7	Assessing the Model.....	34
6	Evaluation	45
7	Findings	46
	References	47
	List of Figures.....	49
	List of Tables	51
	List of Abbreviations	52
	Appendix A.....	53

1 Introduction

Many shops now use self-checkout to speed up the payment process and save workers' time. In a stationary self-checkout, shoppers push their carts to a large machine at the exit, scan each item, and then pay. With a semi-stationary self-checkout, shoppers carry a small scanner while walking through the store and scan each item as they place it in their cart. They pay later at a counter.

These systems are handy because they speed up the payment process for customers, but they pose the risk of customers failing to pay for items by not scanning them. Studies show that approximately five percent of self-scan payments have at least one missing item. It's unclear whether this occurs due to actual fraudulent intent, a simple mistake, or a technical problem with the scanner, but the store still loses money.

The analytical project, 2025, revolves around an established food retailer that has introduced a self-scanning system. This system allows customers to scan items with a handheld mobile scanner. However, the retailer wants to avoid the losses that this system could cause while also avoiding annoying innocent customers. Thus, the project's objective is to build a classification model that can detect whether a scan is fraudulent or not. The model's goal would also be to reduce false positives as much as possible because they cost more and upset customers. If the project achieves these goals, the retailer will be able to save money and operate smoothly and efficiently while maintaining quick and pleasant self-checkout for all.

2 Business Understanding

In every data project, the first step is to understand the business problem, as it lays the foundation for the whole project. This can clearly help us define the project's objects and requirements. For this purpose, we should be able to answer these questions:

1. **What does the retail want?**
2. **Why does the retail want it?**
3. **How can success be defined in this scenario?**
4. **What technology and tools can help such achieve this project plan?**

These answers are crucial in business understanding because it ensures that the data project is truly aligned with requirements and needs of the organisation. Without a deep understanding of the business context, most technically sound solution can end up just missing the mark. First two questions help us clarify the core business problem and their motivation behind it. This can help ensure that we are solving the right problem. Defining success early can help us set measurable goals and evaluation criteria, which can ensure that the project is highly effective. Finally, identifying the right tools and technologies can make sure that the project is feasible and efficient with the existing infrastructure. Hence, these answers help us form the foundation for designing the practical solution for the business, guiding us through all subsequent phases such as data understanding, data preparation, and modelling.

2.1 Background Information

The well-established food retailer now allows customers to scan their own items instead of using a cashier. This concept is called self-checkout. There are two main types of self-checkout. In a stationary self-checkout, shoppers push their carts to a large scanning machine near the exit. They scan every item there and then pay. In a semi-stationary self-checkout, the shopper carries a small scanner while walking through the store. They scan each item as they put it in the cart and pay at a special counter later. Self-checkout is fast and saves staff time, but it also introduces a new problem: missing scans. An item may not be scanned because the customer forgets or wants to cheat, or because the scanner or barcode does not work. Studies show that approximately five percent of bills have discrepancies between the scanned items and the actual items purchased. It is not yet clear how many of these cases are fraud and how many are honest mistakes or technical problems. The store wants to save money while providing good service. Therefore, it needs a smart way to decide which bills to check. Our project will develop a model to classify scans as fraudulent or non-fraudulent. The system will only check a few customers, so most honest shoppers can leave quickly. The system will also learn from new data, making fewer mistakes over time. The shop's success means less money lost from unscanned items, minimal extra checks for honest customers, and clear rules

for staff. With this plan, the store can maintain customer trust, save money, and provide a smooth, fast shopping experience for all.

2.2 Business Objective, Risks and Benefits

The goal of this project is to create a model that can determine whether a self-scan is fraudulent or not. The model does not attempt to explain why a scan is incorrect; it simply labels each bill as fraudulent or non-fraudulent. By identifying risky bills early on, the store can prevent money loss while maintaining quick and convenient self-checkout for honest customers.

A good model benefits the store in several ways. First, it saves money by reducing the number of items that leave the store without being paid for. Second, staff can focus on the few bills marked as fraudulent instead of checking them all, which makes their work easier and faster. Third, when the system works well, shoppers feel that the store is safe and fair. They trust the self-checkout system and continue to use it.

There are also risks that the store must manage. For example, if the model incorrectly labels a non-fraudulent shopper as fraudulent, that customer may become upset and avoid self-checkout in the future. False positives are costly for several reasons: they may cause loyal customers to leave, and they create an added burden for staff and operations. Therefore, a key goal of the model is to minimize false positives while maintaining a high precision rate to ensure that most flagged cases are truly fraudulent. Conversely, if the model fails to detect a fraudulent case (a false negative), the store still loses money. The project must also comply with all privacy laws while analysing shopping behaviour. Additionally, the shop will incur costs for new software and training. The goal is to strike the right balance: detect most real problems, avoid bothering honest customers, and control expenses so the store can protect its earnings while offering a smooth shopping experience.

2.3 Success Criteria and Data Mining Goals

The primary goal of this project is to develop a fraud detection model that accurately identifies fraudulent transactions while minimizing the number of false positives. This ensures that honest customers are not unnecessarily flagged. We will evaluate the model's performance using four key metrics: precision, recall, F1-score, and ROC AUC. Although accuracy is also considered, it is insufficient on its own due to the imbalanced nature of the dataset, as most transactions are non-fraudulent. Precision measures the percentage of flagged transactions that are actually fraudulent, and recall measures the percentage of actual fraud cases that the model detects. The model must achieve a minimum of 0.80 in both precision and recall to be considered successful. This threshold ensures

the model can effectively support fraud prevention efforts without compromising the customer experience at self-checkout.

In addition to these metrics, the ROC AUC score provides insight into the model's ability to distinguish between fraudulent and legitimate transactions across various thresholds. A high area under the curve indicates strong discriminative ability, which is crucial for real-time decision-making. Given the large dataset of over 400,000 records, the model must be efficient and capable of delivering quick results to enable immediate action during customer checkout. Lastly, it is important to monitor the model for signs of overfitting to ensure that it generalizes well to unseen data. The end goal is to deliver a robust, fast, and interpretable model that reduces financial losses, improves operational efficiency, and maintains a seamless, trustworthy self-checkout experience for customers.

2.4 Project Phases and Tool

This project will follow a clear five-step plan called Cross Industry Standard Process for Data Mining (CRISP-DM). Each step is simple yet crucial to building a robust, useful fraud-detection model for the shop. The steps are as follows:

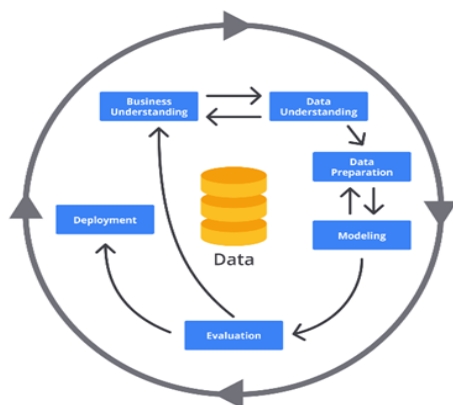


Figure 1. CRISP-DM Model and its 6 Phases

1. Business Understanding

This phase focuses on understanding the business's problem, as well as the project's requirements and objectives. This is the foundation of the entire project. As a result, this phase helps us determine the data mining goals and produce a project plan (Data Science PM, 2024).

2. Data Understanding

This phase focuses on identifying, collecting and analysing the necessary data to accomplish the project goals. We will go deeper into the data set during this phase to identify the existing

relationships among the data. In addition, the data quality will be also verified in this phase (Data Science PM, 2024).

3. **Data Preparation**

This phase is considered the most crucial part of the project. During this phase, the data will be selected, cleaned, and formatted to ensure good data quality for modeling (Data Science PM, 2024).

4. **Modeling**

This phase focuses on selecting the algorithm that will serve as the classification model for our scenario. The test design, including how the data is split into training, test, and validation sets, must also be decided in this phase. For this project, we have decided to try at least three different classification models. Each model will be tested and evaluated to determine which model detects fraud most effectively (Data Science PM, 2024).

5. **Evaluation**

In this phase, the model will be reevaluated to determine if it meets the success criteria and to select the most suitable model for the business. Further deployment steps will also be decided in this phase (Data Science PM, 2024).

6. **Deployment**

In this phase, a plan will be made to deploy the best model, and the model will be trained on the entire dataset. The plan for monitoring and deploying the model should also be decided. Finally, a project review will be conducted to determine what went well and how to improve future projects (Data Science PM, 2024).

The CRISP-DM framework will be used in our project to ensure that a robust and efficient classification model is developed that aligns with the project's objectives, meets business requirements, and delivers actionable insights through a structured and iterative approach.

3 Data Understanding

The goal of the Data-Understanding phase is to move from a broad business idea (“catch as many dishonest baskets as possible while disturbing honest shoppers as little as possible”) to a precise analytical grasp of the raw information that is available. In keeping with CRISP-DM framework, we first describe the available data, then explore relationships and patterns, and finally check the data quality.

3.1 Description of the Data

Variable name	Data format	Variable type	Missing values	Description / Numeric range (or unique values)
trustLevel	Integer	Numerical	none	Internal rating of the customer, 1=lowest trust, 6=highest trust
totalScanTimeInSeconds	Integer	Numerical	none	Total time in seconds between first and last scan • 1 – 1831
grandTotal	Float	Numerical	none	Sum of scanned item prices • 0 – 100
lineItemVoids	Integer	Numerical	none	Count of cancelled scans • 0 – 11
scansWithoutRegistration	Integer	Numerical	none	Trigger pulls with no barcode read • 0 – 10
quantityModification	Integer	Numerical	none	Manual changes of quantity • 0 – 5
scannedLineItemsPerSecond	Float	Numerical	none	Average number of items scanned per

				second • 0.0005 – 30
valuePerSecond	Float	Numerical	none	Average total value of scanned products per second • 0.00 – 99.71 (≈100)
lineItemVoid-sPerPosition	Float	Numerical	none	lineItemVoids divided by total items • 0.00 – 11.00
fraud	Integer	Target	none	Target variable, which can be classified as 1 = fraudulent and 0 = non-fraudulent

Table 1. Data Description

3.2 Further Exploration of the Data

3.2.1 Exploration in Jupyter Notebook

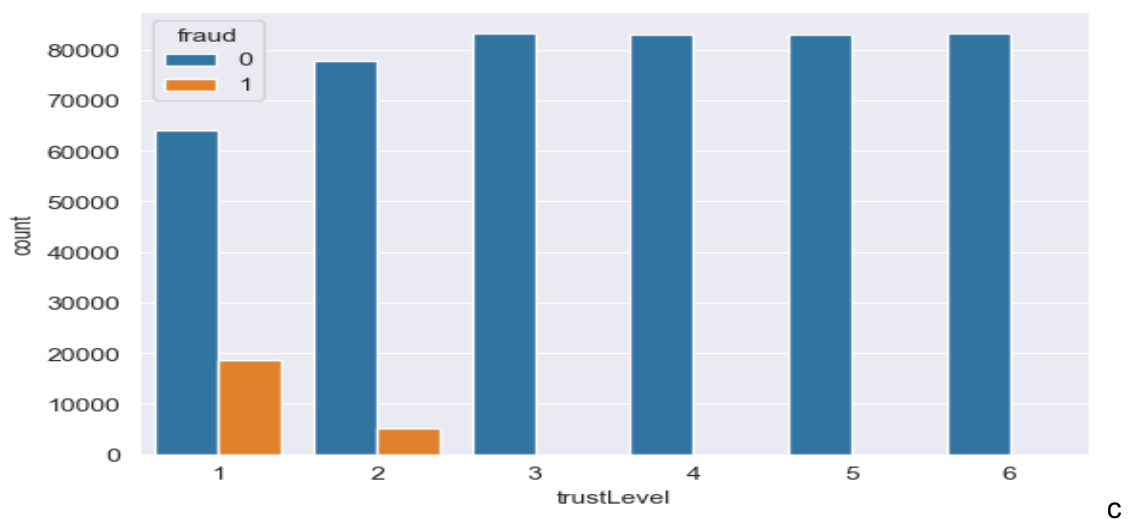


Figure 2. Almost all fraud occurs at trust level 1, with only a small amount at level 2. Baskets from levels 3 through 6 are all free of fraud.

Figure 2 shows a bar chart that illustrates the relationship between trust level and fraud. Trust level is shown on the x-axis and represents the customer's individual trust level. Each bar is colored according to fraud status. At each level, the blue bar (honest baskets) is the tallest. However, level 1 stands out because the red bar (fraudulent baskets) reaches a significant portion of the total: about one-quarter of all level 1 transaction. Moving to level two, the red bar drops to a lower level. From levels three to six, no fraudulent data is seen. The message is clear: existing trust scores already isolate almost all risk in the lowest two tiers. Therefore, any real-time fraud checks can focus on lower-trust customers without wasting effort on higher-trust customers.

As a result, we will reduce the dataset to the first two trust levels because fraud is only seen at these levels. Trust levels 3 and above can be automatically classified as non-fraudulent.

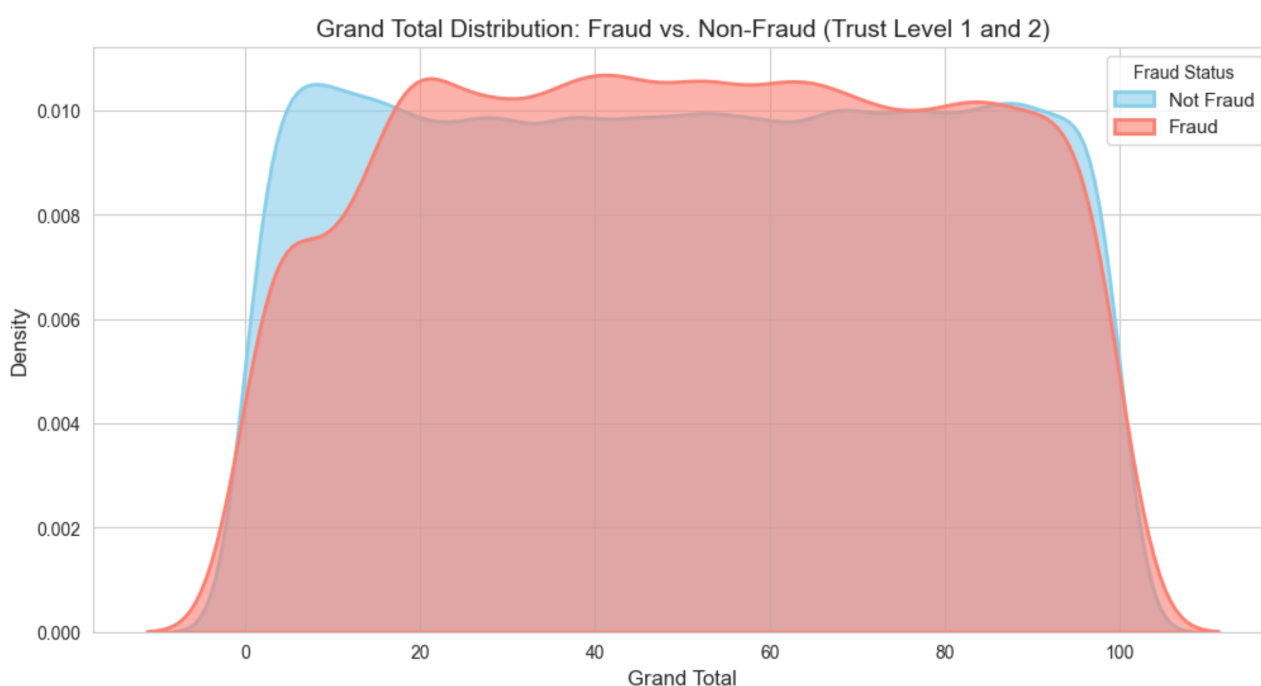


Figure 3. Comparison of Grand Total Distribution between Fraudulent and Non-Fraudulent cases

As shown in the graph above, there isn't a significant difference in the total value of baskets between fraud and non-fraud cases. The red and blue curves cover nearly the same range of totals, from low to high. This suggests that fraud occurs across the board, not just in high- or low-value transactions. Fraudsters seem to blend in when it comes to their overall spending. This means that relying only on the grand total isn't an effective way to detect fraud. Instead, it could be a more effective detector when used with other parameters, such as the number of items or canceled scans, where patterns

reveal a clearer picture. Although the total basket value doesn't clearly distinguish between fraud and non-fraud, fraudsters seem to operate within a "safe" middle range, neither spending too little nor too much. This makes the grand total more useful as part of a combined check rather than as a standalone signal.

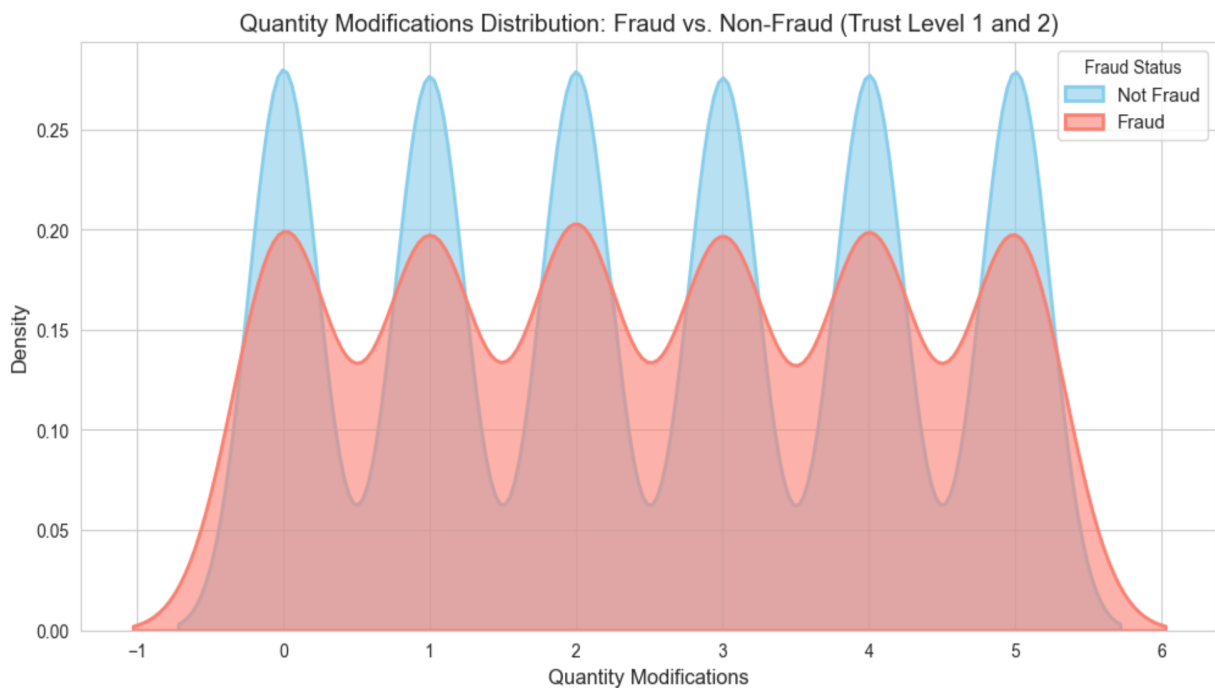


Figure 4. Comparison of Quantity Modifications Distribution between Fraudulent and Non-Fraudulent cases

Figure 4 compares how often quantity modifications occur in fraud and non-fraud baskets, and the difference is clear. Honest transactions (the blue line) show strong, sharp spikes at each whole number. This is especially evident at zero, suggesting that most regular customers don't alter quantities significantly, if at all. In contrast, the red curve (fraud) is more evenly spread out and has fewer peaks, suggesting that fraudsters are more likely to change item quantities during checkout. This pattern implies that quantities are more common and random in fraudulent activity. While a few honest customers might change a quantity here or there, fraudsters tend to make these modifications more frequently, possibly as part of a trick to manipulate totals or mask removed items. Quantity modification alone isn't a perfect fraud indicator, but it is useful when combined with other signals, such as voids, item count, or trust level. A basket with multiple quantity changes and other risky behaviours could be a stronger candidate for fraud than one with just a high item total.

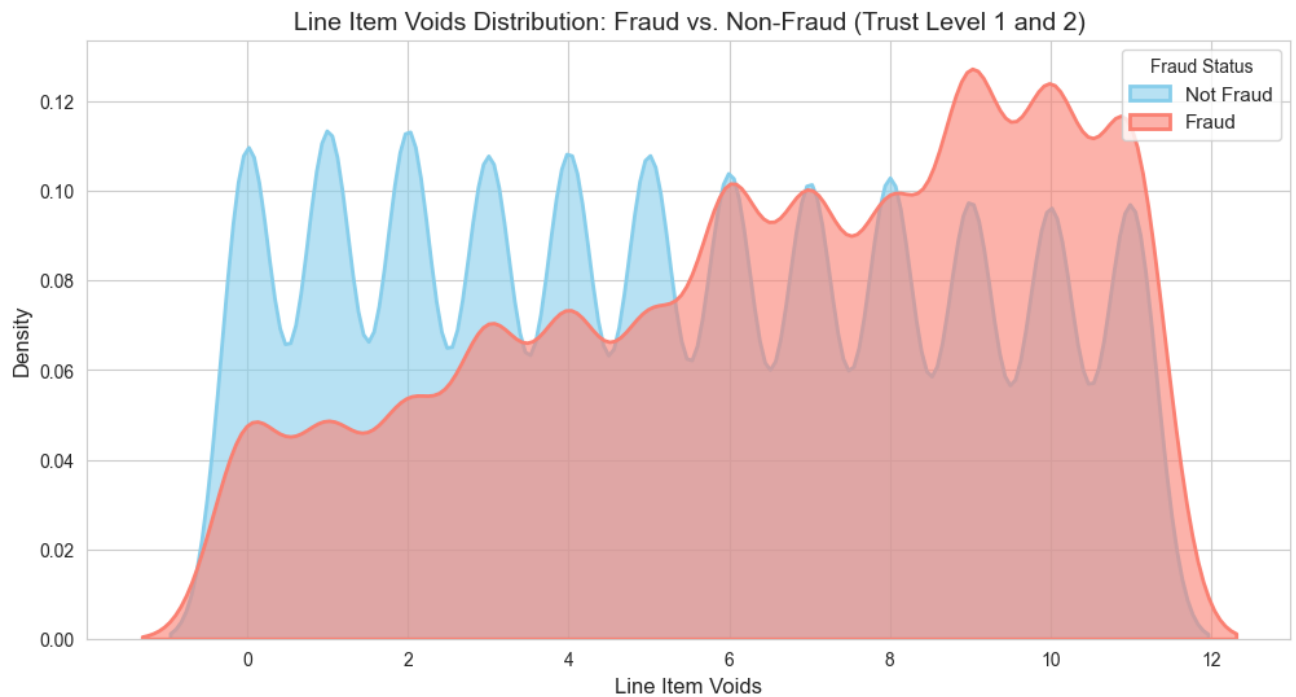


Figure 5. Comparison of Line Item Voids Distribution between Fraudulent and Non-Fraudulent cases

The graph shows that the blue curve, representing honest baskets, is highest between zero and five cancelled scans, after which it falls quickly. The red curve, which shows fraudulent baskets, stays low initially but climbs once the void count reaches six and remains high up to twelve. This pattern confirms that a high number of voids is a strong warning signal, even when limiting the data to the riskiest shoppers. For practical purposes, stores can implement a simple rule: if a Trust Level 1 or Trust Level 2 customer cancels six or more items, send the basket to a follow-up check. Combining this rule with other live signals, such as empty scans and slow scanning, should catch most fraud while disturbing only a few honest customers.

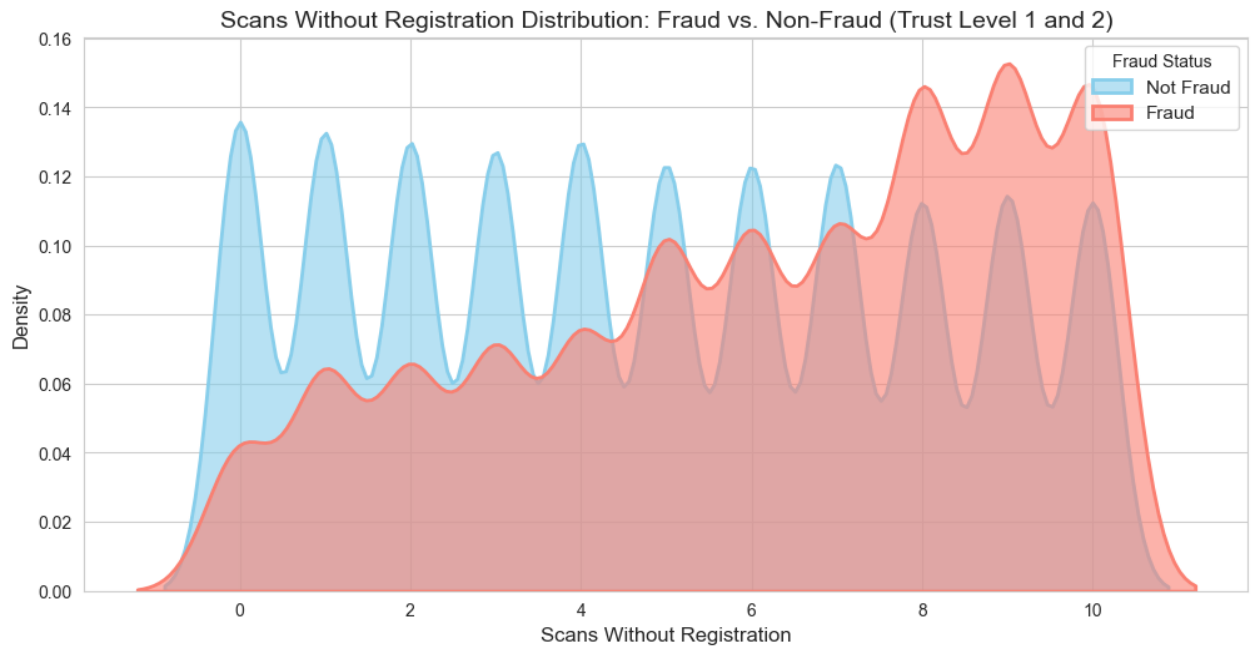


Figure 6. Comparison of Scans Without Registration Distribution between Fraudulent and Non-Fraudulent cases

In the plot, the blue curve represents honest baskets, which cluster tightly between zero and five missed scans and drop off sharply beyond six. This shows that genuine shoppers rarely allow many items to slip by the scanner. The red curve represents fraudulent baskets. It stays low initially, but begins to climb around four missed scans. It overtakes the blue curve near seven missed scans and peaks around nine or ten. Therefore, when a trust-level-1 or -2 customer misses seven or more scans, the basket is likely fraudulent. Flagging those baskets for a quick check, especially if other suspicious activity is present, such as lots of voids or slow scanning, should catch most cheaters while inconveniencing only a few honest customers.

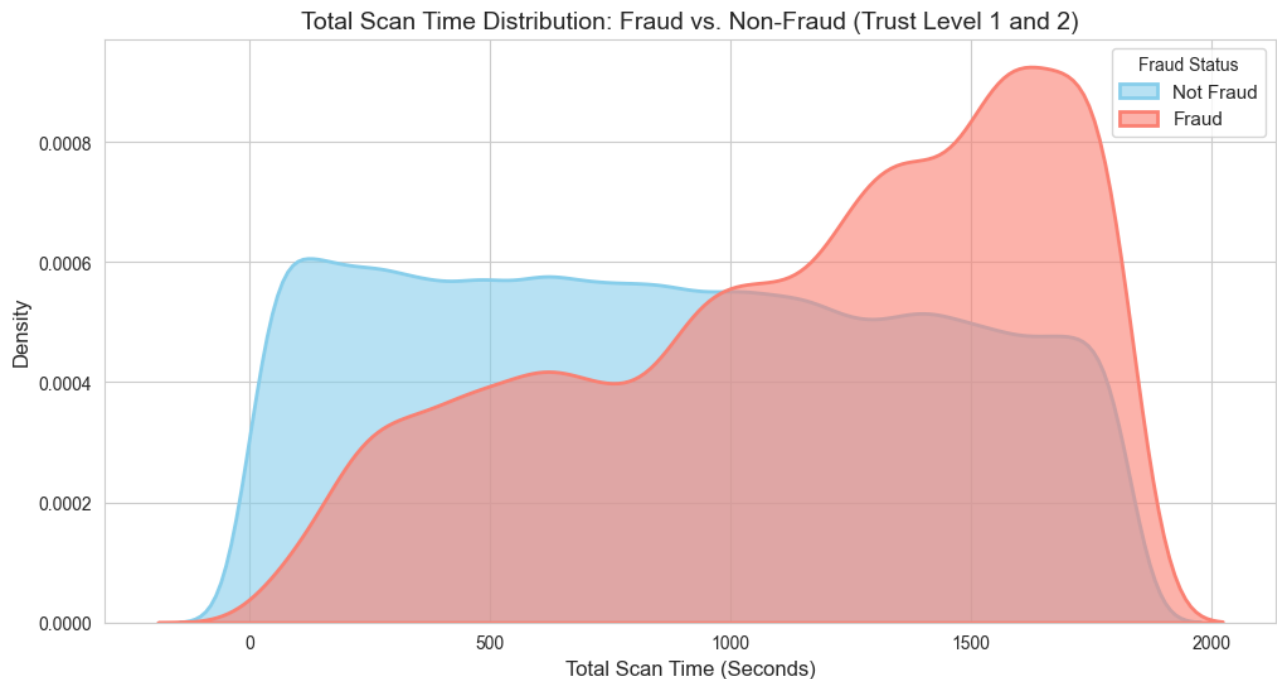


Figure 7. Comparison of Total Scan Time Distribution between Fraudulent and Non-Fraudulent cases

The blue curve in *Figure 7* shows how long genuine shoppers spend at the shop. Most wrap up in 150–900 seconds, and the density falls sharply after 1,000 seconds. The red curve, which tracks fraud cases, remains low in the quick-scan zone but steadily rises beyond 900 seconds, peaking around 1,500–1,700 seconds before tapering off near 2,000 seconds. In other words, slow checkouts are a strong warning signal. Once a low-trust customer's total scan time exceeds about 1,000 seconds, the likelihood of fraud surpasses the likelihood of honest behavior. It is important to flag those long sessions, especially when paired with other clues, such as many voids or missed scans.

These four discoveries already give us a clear recipe for the next step. We now see that the shop's `trustLevel` is the first filter and that three live signals—many line-item voids, many empty scans, and an unusually long scan time—clearly separate risky baskets from normal ones. During the data preparation phase, we can maintain these raw counts and establish straightforward cutoff rules, such as voids greater than or equal to six or empty scans greater than or equal to eight. Additionally, we can incorporate supplementary features that integrate `trustLevel` with each error counter.

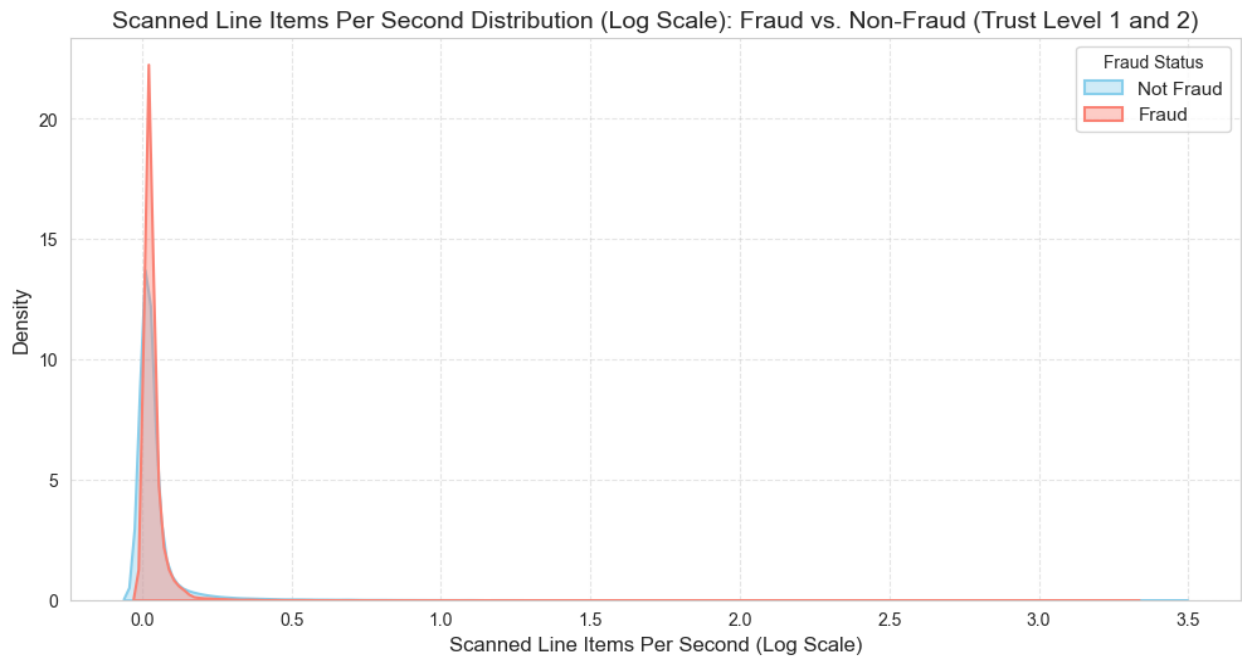


Figure 8. Comparison of Scanned Line Items Per Second Distribution between Fraudulent and Non-Fraudulent cases

The blue and red density curves in Figure 8 overlap near the lowest end of the log-scaled x-axis and then fade away together. Both honest shoppers and fraudsters work through their baskets at roughly the same speed, between 0.03 and 0.05 items per second. Neither group shows a second hump at faster speeds. Since the two lines overlap so closely, "scanned line items per second" does not help distinguish risky baskets from safe ones once filtered to the lowest trust levels. This metric can remain in the dataset for completeness, but it has little separating power compared to void counts, missed scans, or total scan time.

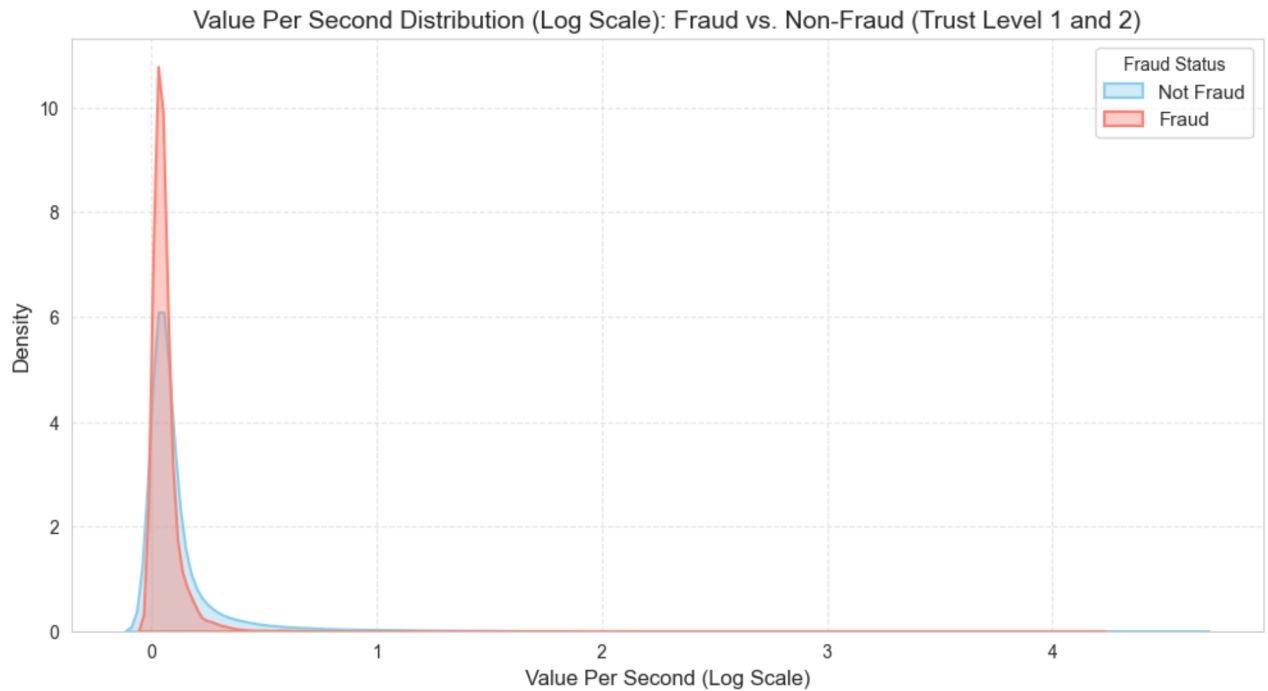


Figure 9. Comparison of Value Per Second Distribution between Fraudulent and Non-Fraudulent cases

The plot above is similar to Figure 8 in terms of the results it yields. The blue and red curves represent honest and fraudulent baskets, respectively, and show the average total value of scanned products per second ("value per second") on a log scale. Both curves peak sharply at low values, between 0 and 0.2, indicating that honest shoppers and fraudsters scan products at similar rates in terms of monetary value per second. The curves overlap almost entirely, with no clear separation or distinct pattern at higher speeds. Therefore, "value per second" does not distinguish fraud from non-fraud cases among the riskiest shoppers (trust levels 1 and 2). Again, this metric can remain in the dataset for completeness, it offers little separating power compared to stronger features, such as line item voids, missed scans, and unusual quantity modifications.

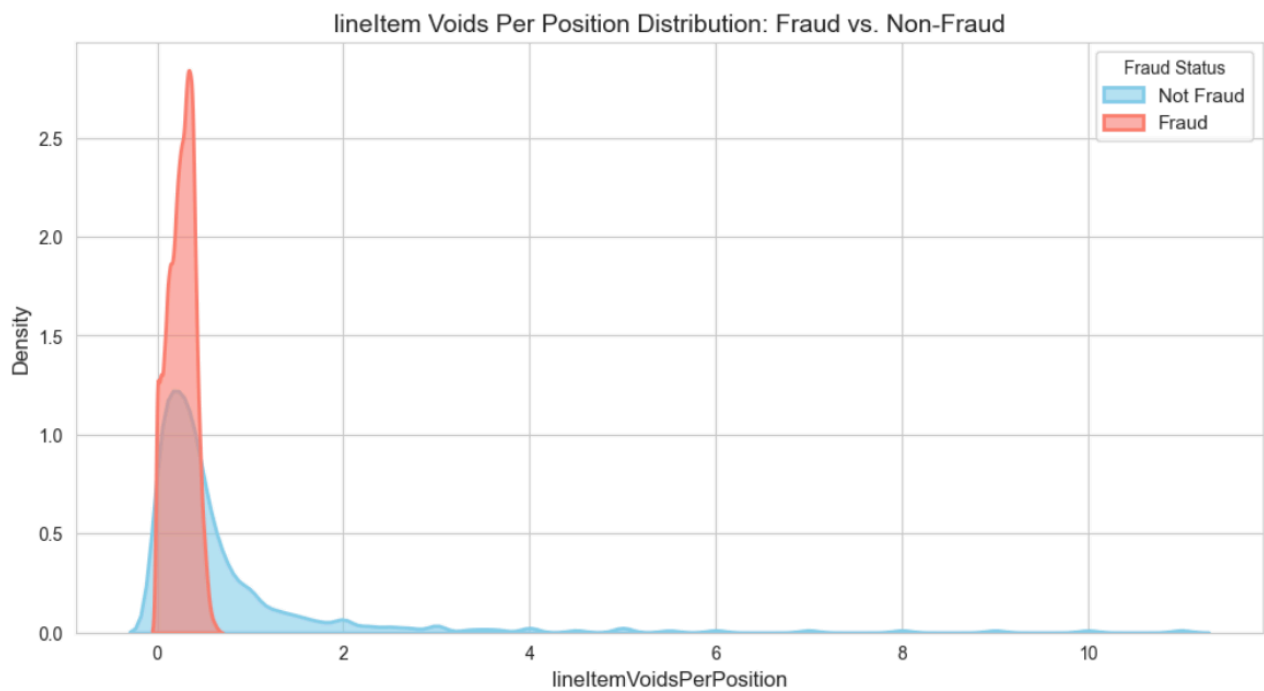


Figure 10. Comparison of Line-Item Voids Per Position Distribution between Fraudulent and Non-Fraudulent cases

This graph illustrates the frequency of line-item voids per scanned position. There is a clear difference between fraud and non-fraud cases. Honest baskets have a broader spread, allowing some shoppers to void multiple items per position, though this happens less frequently. Conversely, the red curve is tall and narrow with a sharp peak close to zero. This indicates that most fraudulent baskets have a small number of voids per position, but these voids are consistent. This tells us that, although both fraudsters and honest shoppers cancel items, fraud cases tend to concentrate around a specific voiding pattern. Fraudsters appear to keep the number of positions low while still voiding items, likely as a way to manipulate or remove key items without drawing too much attention. It is assumed that a higher frequency of voids in relation to the total number of scanned items can be a useful fraud indicator. When the rate of voids per position rises above the normal range seen in honest shopping behaviour, it may be worth investigating further. This metric is especially powerful when paired with other indicators, such as trust level, total void count, or session duration.

3.2.2 Exploration in Orange

We decided to use Orange for data exploration because it is easy-to-use, low-code software (Orange Data Mining, 2025). For this exploration, we took a sample of 25,000 data points. The first step we took in Orange was to check the correlation between the variables, as shown in Figure 11. We found that the most common correlation occurs between `ScannedLineItemsPerSecond` and `ValuePerSecond`.

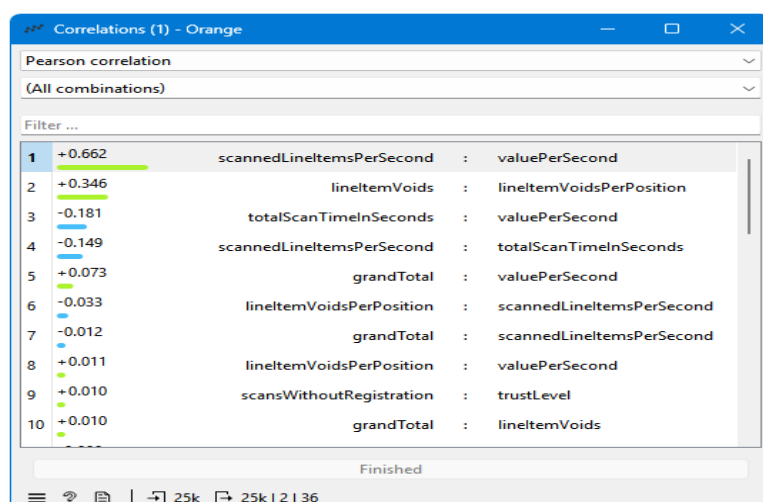


Figure 11. Correlation between variables

Furthermore, we performed an analysis in Orange with Violin Pilot that was similar to the analysis performed with a density plot in the Jupyter notebook. The findings regarding the plot were identical. Figure 12 below shows an example of a violin plot executed in Orange. Additionally, Orange showed us that there is no fraud in the trust level from 3 to 6.

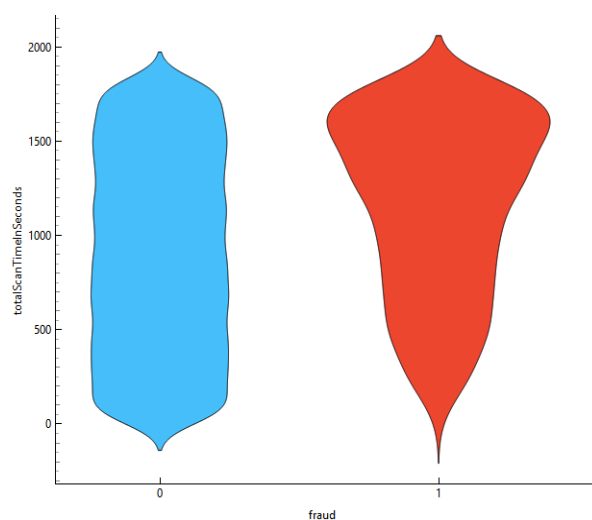


Figure 12. Violin plot of the relationship between totalScanTimeInSeconds and the fraud variable

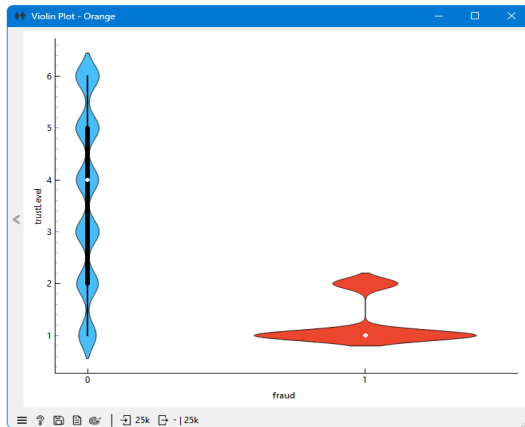


Figure 13. Violin plot of the relationship between *trustLevel* and the *fraud* variable

3.3 Verification of the Data Quality

To determine whether the data was clean or dirty, we decided to use Pandas, a great data manipulation and analysis library (Pandas, 2024). First, we checked for null values by executing the Python statement “`df.isnull().sum()`”, which showed that there were no null values, as seen in *Figure 14*. To confirm this, we ran the statement “`df[df.isnull().any(axis=1)]`” to see if any rows contained NaN values. This ensured us that the data was quite clean for the initial analysis. Additionally, to better understand the dataset, we executed different statements, such as “`df.describe()`”, to gain a more detailed statistical summary of the data.

```
# checking for null values
df.isnull().sum()
#data looks quite clean for the initial analysis
[4]
```

trustLevel	0
totalScanTimeInSeconds	0
grandTotal	0
lineItemVoids	0
scansWithoutRegistration	0
quantityModifications	0
scannedLineItemsPerSecond	0
valuePerSecond	0
lineItemVoidsPerPosition	0
fraud	0
dtype: int64	

Figure 14. Code snippet executed in Jupyter Notebook checking for null values

4 Data Preparation

As stated earlier, the data preparation phase will focus on data selection, which will be done in our case through model-based feature selection. Since the data is clean according to our initial analysis, we will leave it as is and return to cleaning if necessary, according to the chosen classification algorithm. We created two new variables from the original dataset, which will be explained in detail in the construct data section. Since only the data sets with trust levels 1 or 2 contain fraud, we decided to reduce the data set size by excluding data with trust levels 3 or higher. Section 4.6 of the paper will explain the necessary formatting applied to the data.

4.1 Missing Values (NA) and Duplicates

From our initial analysis in both Jupyter Notebook and Orange, it appears that the data set does not contain any missing values. We then searched for duplicates using the Unique Widget and found none. Thus, the data is ready to use.

4.2 Construct Data

The goal of construct data would be to derive new variables from the original data set, that would be helpful in the classification of fraud or not. In our case, we were able to identify two variables that could be handy in detecting whether the scan is fraudulent or not. The first variable, we constructed is called “totalItems”. Total Items is derived by multiplying “totalScanTimeInSeconds” with “scannedLineItemsPerSecond”. To explain the impact of this new variable, we have created a stacked density plot across the variable “fraud”, as shown in

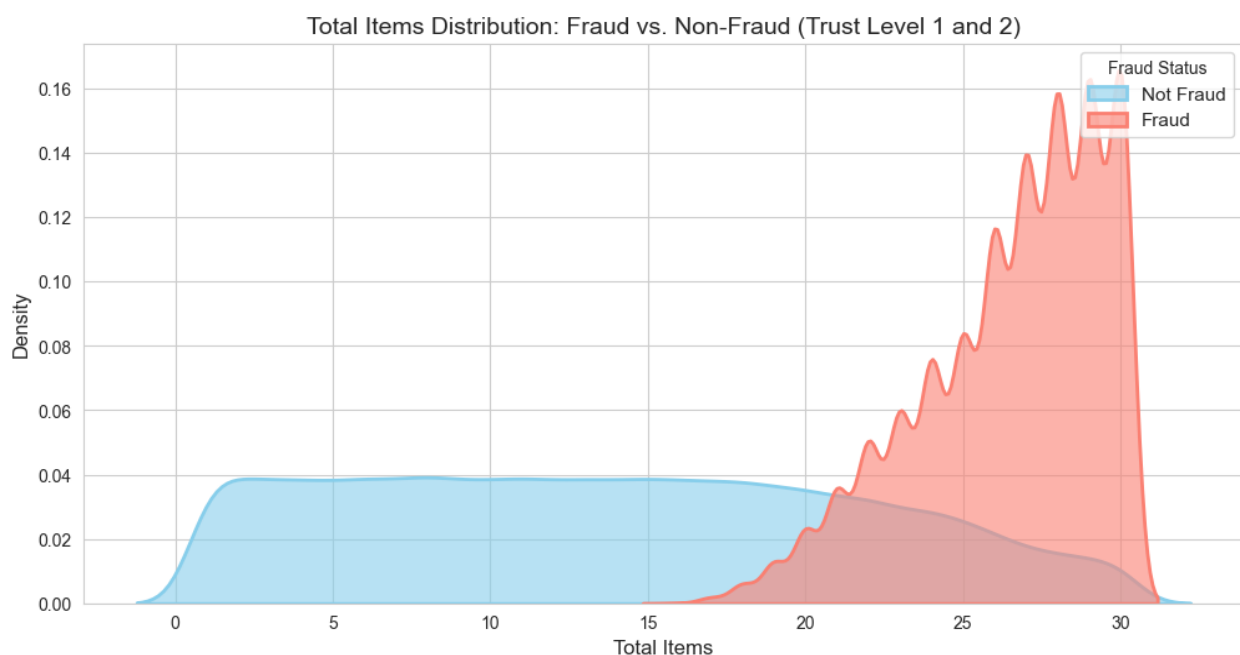


Figure 15. Comparison of Total Items Distribution between Fraudulent and Non-Fraudulent cases

The blue curve (honest baskets) remains relatively flat from one to approximately eighteen items, then tapers off. This indicates that genuine shoppers typically purchase a limited number of products, rarely exceeding twenty-five. The red curve (fraudulent baskets) is negligible at low counts, but it climbs sharply after twenty items and peaks near thirty. In that range, it towers over the blue curve. In other words, a large basket is a clear danger sign among the lowest-trust shoppers. When the total number of items exceeds the low twenties, the odds of fraud quickly overtake the odds of innocence. Adding a rule such as "flag any Trust-1 or Trust-2 basket with more than twenty-two items" to existing checks for voids, missed scans, and long session times would improve the store's real-time fraud detection without bothering most honest customers. Therefore, the newly created variable "Total Items" can be a great asset in determining whether a scan is fraudulent.

The second variable we constructed is called "AverageValuePerItem," which is derived by dividing "grandTotal" by "totalItems." As shown in the figure below, the impact of this variable can be explained with the help of a stacked density plot across the variable "fraud."

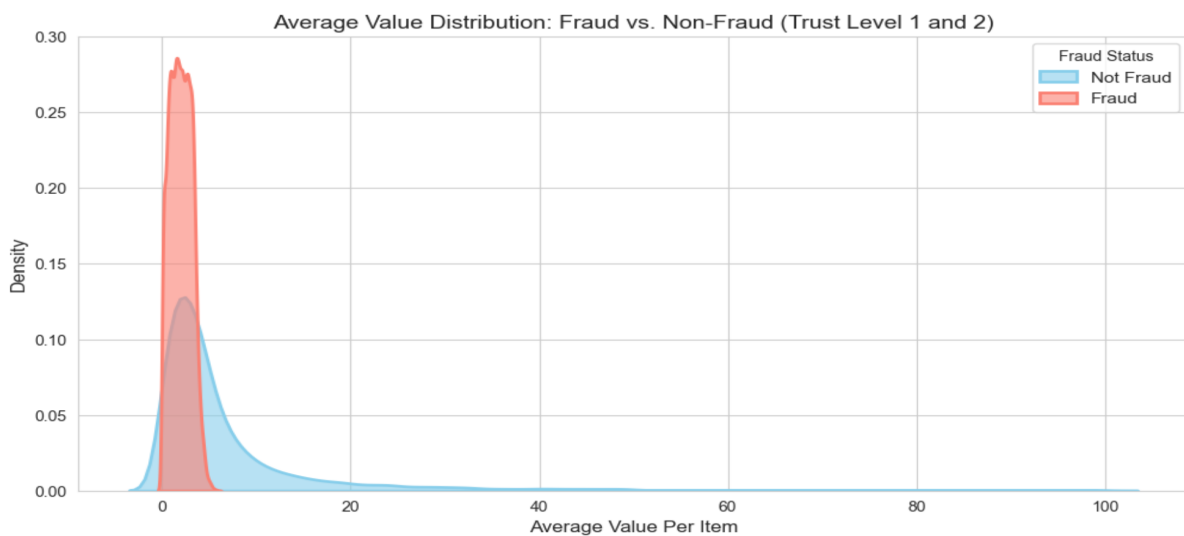


Figure 16. Comparison of Average Value Distribution between Fraudulent and Non-Fraudulent cases

The resulting plot clearly shows a pattern. Fraudulent transactions exhibit a narrowly concentrated distribution, with the average item value clustered just above zero. This suggests that fraudsters tend to select a large number of low-cost items per transaction, likely as a strategy to avoid triggering alarms based on the value of an individual item. The minimal spread in the red curve further supports this, indicating a consistent behavioral pattern among fraudsters. In contrast, non-fraudulent transactions show a much wider range. The blue curve spreads out more, meaning honest customers tend to have more confidence in their purchases. Some of their baskets include items with a much

higher average value. This makes sense, as regular shoppers may purchase both inexpensive everyday items and more expensive products depending on their needs or the occasion.

These findings suggest that fraud is not necessarily driven by high-value goods but rather by volume, which often involves numerous low-cost items. From a fraud detection standpoint, this insight is highly actionable. This finding is useful for spotting fraud automatically. The average value per item can be a helpful clue for systems trying to detect suspicious transactions. For example, if someone buys many items, but each one is inexpensive, it might be worth taking a closer look. This pattern matches what we've seen in many fraud cases.

4.3 Balancing Data Set

The original dataset is highly imbalanced, with 474,394 non-fraud cases and only 23,727 fraud cases, with fraud making up just around 5% of the data. Initially, we considered using SMOTE from the imbalanced learning (imblearn) library to oversample the minority class, that is the fraud class (Imbalanced learn, 2025). However, after discussing the matter during our coaching session, we concluded that the dataset was large enough to train an effective classification model without relying on synthetic data.

We then experimented with undersampling the majority (non-fraud) class to achieve balance. However, this approach performed poorly on real-world test data. In practice, fraud detection datasets remain imbalanced. By undersampling, we would have discarded a significant amount of valuable non-fraud data that could help the model learn more subtle fraud patterns.

Consequently, we decided against artificially balancing the dataset. Instead, we retained the original class distribution by setting `stratify=y` during the train-test split. To address class imbalance during model training, we employed a cross-validation approach with class weighting. This approach enables the model to focus on the minority class while retaining critical information from the majority class.

4.4 Reducing categories / grouping

As we previously noted during the exploratory data analysis, fraudulent transactions only occur within trust levels 1 and 2. Therefore, we reduced the dataset by excluding trust levels 3 and above since they contain only non-fraudulent data. Transactions from these higher trust levels can be confidently classified as non-fraudulent and do not provide useful information for fraud detection. This filtering significantly reduces the size of the dataset, making the modeling process more efficient without losing relevant fraud data. The dataset can be filtered in Python using the following statement: `df_filtered = df[df['trustLevel'] <= 2]`. The class distribution is now as follows: non-fraud (0): 141,899, and

fraud (1): 23,727. This refined dataset ensures the model focuses on trust levels where fraud actually occurs, improving efficiency and relevance.

4.5 Feature Selection

To begin the feature selection process, we checked for highly correlated features that were more than 95% similar to each other. The goal was to eliminate features that essentially provide the same information, as this can confuse the model and negatively impact performance. We used a correlation matrix to identify these features, but none of them were closely related. In the end, therefore, no features were removed at this step. The code snippet below illustrates how this was executed in Python.

```
# --- 2. Remove highly correlated features ---
correlation_matrix = X.corr().abs()
upper = correlation_matrix.where(np.tri(correlation_matrix.shape[0], k=-1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]

print("\nCorrelated Features Analysis:")
print("Features to drop due to high correlation (>0.95):", to_drop)
X_unrelated = X.drop(to_drop, axis=1)
print("Shape after removing correlated features:", X_unrelated.shape)
```

Figure 17. Code snippet executed in Jupyter Notebook to remove highly correlated features

Since we had already chosen the models to use, we took a more balanced approach to feature selection. Rather than relying on just one model's perspective, we ran a feature importance analysis across all four models. Running the analysis across all four models would also help us avoid model bias. This allowed us to see which features were consistently important, regardless of the model. To keep it fair and unbiased, we then averaged the importance score for each feature. This way, we could confidently select the most valuable features based on input from all models, not just one.

In the case of feature importance, we decided to use an 80/20 ratio to prepare the data for training and testing. This means that 80% of the data is used to train the model and the remaining 20% is used to evaluate its performance. This approach provides the model with ample information to learn from while retaining enough data to accurately evaluate its performance.

```
# --- 3. Split data (stratified, to preserve fraud/non-fraud ratio) ---
X_train, X_test, y_train, y_test = train_test_split(
    X_unrelated, y, test_size=0.2, stratify=y, random_state=42
)
```

Figure 18. Code snippet executed in Jupyter Notebook to split data

We also used stratified sampling (Stratify = Y) to ensure that the ratio of fraud and non-fraud cases remained consistent between the training and test sets. This is especially important in the field of

fraud detection, where fraud cases are much less common. Without this, the test set might have too few fraud examples, which would produce misleading results because the model could become biased toward non-fraud due to its prevalence. Finally, we set `random_state` to 42 to ensure that the split is always performed consistently each time the code is run. This helps keep our results consistent and makes it easier to debug or compare different models.

As we have used Logistic Regression and Support Vector Machine (SVM) models, we have applied `StandardScaler` because these models are quite sensitive to the scale of the input features. Both models rely on calculations such as distances or coefficients. If one feature has much larger values than the others, it can end up having too much influence on the model, even though it is not actually more important. Using `StandardScaler` ensures that all features are on the same scale, with a mean of 0 and a standard deviation of 1, which helps the models work more efficiently and fairly. Therefore, no feature will dominate simply because of its scale. It also helps the models converge faster and improves performance overall.

4.5.1 *Random Forest Classifier*

Random Forest ranks features based on how much Gini impurity or entropy they reduce across trees. This means that it favors features with consistent splitting power across decision trees. The top 10 features for the Random Forest model are shown below.

1. `totalItems`
2. `AverageValuePerItem`
3. `lineItemVoidsPerPosition`
4. `trustLevel`
5. `scannedLineItemsPerSecond`
6. `totalScanTimeInSeconds`
7. `scansWithoutRegistration`
8. `lineItemVoids`
9. `valuePerSecond`
10. `grandTotal`

4.5.2 *XGBoost Classifier*

XGBoost uses gradient boosting, which means it builds trees sequentially. Importance is based on how often and how early a feature is used to reduce error. Thus, the top 10 features for XGBoost are as follows:

1. `totalItems`
2. `trustLevel`
3. `lineItemVoids`
4. `scansWithoutRegistration`
5. `totalScanTimeInSeconds`

6. valuePerSecond
7. grandTotal
8. scannedLineItemsPerSecond
9. lineItemVoidsPerPosition
10. AverageValuePerItem

4.5.3 Logistic Regression

Logistic regression relies entirely on linear relationships, and importance is based on the scaled absolute value of the coefficients. The top 10 features based on logistic regression are as follows:

1. totalItems
2. lineItemVoidsPerPosition
3. AverageValuePerItem
4. trustLevel
5. lineItemVoids
6. totalScanTimeInSeconds
7. scansWithoutRegistration
8. grandTotal
9. scannedLineItemsPerSecond
10. valuePerSecond

4.5.4 SVM

SVM ranks importance by weight in the hyperplane definition, which is often quite similar to Logistic Regression when linear. The top 10 features are:

1. totalItems
2. lineItemVoidsPerPosition
3. trustLevel
4. AverageValuePerItem
5. lineItemVoids
6. totalScanTimeInSeconds
7. scansWithoutRegistration
8. grandTotal
9. valuePerSecond
10. scannedLineItemsPerSecond

We have now calculated the average score, or mean, from all models to determine the top ten features. This method selects the best variables in a fair and balanced way because it is not based on a single model. Figure 18 shows the variables with their combined feature scores. The top ten features would be selected while training the model for classification. In this case, the variable "valuePerSecond" would be dropped. These top ten features will improve our model's effectiveness in real-life fraud detection.

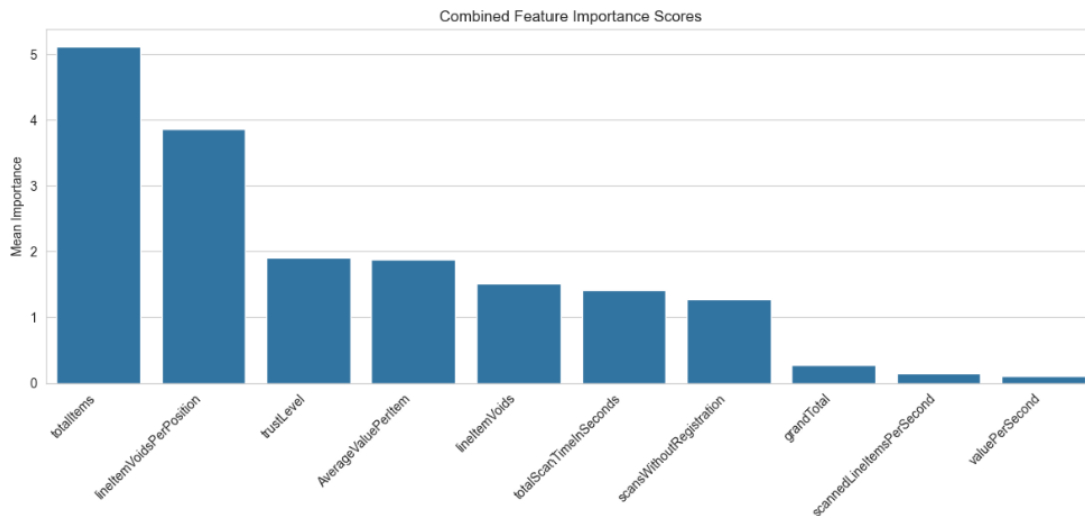


Figure 19. Combined feature importance scores from all models

4.6 Formatting the Data

As previously mentioned, StandardScaler was used to format the data for use in models such as Logistic Regression and SVM. StandardScaler is a preprocessing technique that standardizes the features of a dataset. It transforms the data so that each feature has a mean of zero and a standard deviation of one. This is important because machine learning algorithms, such as Logistic Regression and Support Vector Machines, are sensitive to the scale of input features.

Feature scaling is a fundamental preprocessing step in machine learning, especially for algorithms that are sensitive to the magnitude of input variables. Models such as logistic regression and SVM depend on calculations involving distances or dot products between feature vectors. Consequently, these models can be significantly influenced by the scale or units of the input features. If the range of features varies widely, those with larger values could affect the model's performance, even if they are not inherently more important. For example, consider two features: totalScanTimeInSeconds, which can reach values up to 1,831, and valuePerSecond, which typically does not exceed 100. Without scaling, the model may assign greater importance to totalScanTimeInSeconds solely due to its larger numerical range. This bias can lead to suboptimal model behaviour and inaccurate predictions.

Thus, Standardization techniques, such as StandardScaler, are applied to find a solution to this issue. This method, as stated, transforms each feature so that it has a mean of zero and a standard deviation of one. This ensures that all features contribute equally during the model training process, thereby improving the fairness and effectiveness of the algorithm. Scaling improves model accuracy

and convergence and ensures that the learning process reflects true feature relationships rather than being distorted by units or scales.

5 Modeling

The modeling phase of our analytics project consists of four steps: selecting the classification algorithm, generating the test design, building the classification model, and assessing the model based on domain knowledge. For our scenario, we chose four different algorithms: XGBoost Classifier, Random Forest Classifier, Logistic Regression, and SVM. The following subsection of the paper explains the models and the reasons for choosing them.

5.1 XGBoost Classifier

5.1.1 *Understanding the XGBoost Classifier*

The XGBoost classifier is a machine learning algorithm based on the eXtreme Gradient Boosting (XGBoost) framework. Specifically designed for classification problems, it builds an ensemble of decision trees where each subsequent tree is trained to correct the errors made by the previous ones. Through this learning process, the model can make highly accurate predictions, especially in scenarios like ours, where the data is structured (XGBoost Documentation, n.d.).

5.1.2 *Why is the XGBClassifier suitable for our case?*

1. It works well with imbalanced data.

In many real-world classification tasks, one class is often significantly underrepresented compared to others (e.g., fraud detection or rare disease diagnosis). The XGBoost classifier addresses this issue by providing a `scale_pos_weight` parameter that allows the model to emphasize the minority class more during training. This helps prevent the classifier from being biased toward the majority class and improves performance in identifying rare outcomes (XGBoost Documentation, n.d.).

2. Combines Weak Learners:

XGBoost uses decision trees as weak learners that improve iteratively through a process called boosting. While a single tree may not perform well, together they form a strong predictive model. This approach enables the classifier to identify complex patterns and non-linear relationships in the data, thereby enhancing its overall accuracy (XGBoost Documentation, n.d.).

3. Learns well via Gradient Boosting

This model relies on gradient boosting, which minimizes errors via gradient descent optimization. This method ensures that each new model reduces the overall model's loss. This optimization strategy contributes to XGBoost's reputation for strong predictive performance in academic and industrial settings (XGBoost Documentation, n.d.).

4. Works well with structured data

The XGBoost Classifier is particularly well-suited for structured datasets where features are clearly defined in a tabular format, such as in our project. It is optimized for memory efficiency, speed, and flexibility in handling sparse and missing values, making it ideal for many practical data science applications (XGBoost Documentation, n.d.).

5. Easy to Interpret:

Although ensemble models are often considered "black boxes," XGBoost provides users with tools that allow them to interpret model behavior in detail. Features such as feature importance scores and tree visualizations help users understand which variables most contribute to the model's decisions. This is crucial in fields where interpretability is as important as accuracy (XGBoost documentation, n.d.).

5.1.3 How Can We Use It in Our Project?

Since our project involves structured data with imbalanced classes, XGBoost's Classifier is ideal for effectively handling the imbalance. By tuning its parameters, especially `scale_pos_weight`, we can ensure that underrepresented classes receive adequate attention. The XGBClassifier's high performance, ability to model complex relationships, and interpretability features make it ideal for building a reliable, explainable classification model for our use case.

5.2 Logistic Regression

5.2.1 Understanding Logistic Regression

Logistic regression is a classification algorithm provided by the Scikit-learn library that predicts categories based on given data. It uses the logistic function to model the probability that a given input belongs to a particular class. This approach is effective for binary classification tasks. The model estimates the parameters (coefficients) that maximize the likelihood of observing the given data. This makes logistic regression a powerful choice for various classification scenarios (scikit-learn, n.d.).

5.2.2 Why Is Logistic Regression a Good Selection for This Case?

1. Effective for Classification Tasks

Logistic Regression is specifically tailored for classification problems. It provides a probabilistic framework that allows for class predictions and estimation of the probability of each class. This probabilistic output is valuable in applications where understanding prediction confidence is crucial (scikit-learn, n.d.).

2. Handles Large Datasets Efficiently

The algorithm is computationally efficient and can effectively handle large datasets. Therefore, it is a very good choice for a dataset as large as the one in that project. Scikit-learn's implementation supports various optimization algorithms. This flexibility ensures that Logistic Regression can be applied to large-scale problems without experiencing significant performance issues (scikit-learn, n.d.).

3. Provides Probabilistic Outputs

One of the key strengths of logistic regression is its ability to provide probability estimates for each class. This feature is especially useful in scenarios where the certainty of a prediction is as important as the prediction itself. For instance, in medical diagnosis or risk assessment, knowing the probability of a condition can lead to better decision-making (scikit-learn, n.d.).

5.2.3 How We Can Use It in Our Project

In our project, which involves classifying data points into distinct categories, LogisticRegression is a reliable baseline model. Its simplicity and interpretability make it an ideal starting point for developing models. Analyzing the coefficients allows us to understand the influence of each feature on the outcome, aiding in feature selection and helping us to understand the underlying data patterns. Furthermore, the model's probabilistic outputs can prioritize cases based on prediction confidence, which is valuable for allocating resources and making decisions.

5.3 Support Vector Machines

5.3.1 Understanding SVM

A Support Vector Machine (SVM) is a robust supervised learning algorithm used for classification and regression tasks. It identifies the optimal hyperplane that separates data points from different classes with the maximum possible margin. The goal is to maximize this margin as much as possible by relying on critical samples, such as support vectors. This approach ensures strong performance, especially in high-dimensional datasets where class separation is clear (Pedregosa et al., 2011).

5.3.2 Why Is SVM a Good Selection for This Case?

1. High Performance in Complex Spaces

SVM is particularly well-suited for high-dimensional, nonlinear datasets. By applying kernel functions, SVM can transform input data into higher-dimensional feature spaces, allowing for linear separation. This makes SVM highly effective at uncovering complex patterns and class boundaries (Pedregosa et al., 2011).

2. Effective with Limited Data

A SVM often performs well also with smaller datasets because it focuses on the most informative data points, which are likely to be the support vectors used to construct the decision boundary. This property is especially useful when labeled training data is limited (Pedregosa et al., 2011).

3. Flexibility Through Kernel Trick

One of SVM's core strengths is its flexibility via the kernel trick. This trick allows various kernel functions, such as linear, polynomial, and radial basis function (RBF), to be used without explicitly transforming the feature space. This enables SVM to adapt to linearly and nonlinearly separable data (Pedregosa et al., 2011).

5.3.3 *How We Can Use It in Our Project*

In our project, the goal is to classify data into specific categories. SVM is a compelling choice because it can handle complex, high-dimensional data. Leveraging different kernels allows us to tailor the model to our dataset's structure, potentially achieving better classification results. Furthermore, analyzing the support vectors provides insights into the most influential data points, informing data collection and feature selection in future iterations (Pedregosa et al., 2011).

5.4 Random Forest Classifier

5.4.1 What is a RandomForestClassifier?

A RandomForestClassifier is a machine learning model that uses an ensemble of decision trees to perform classification tasks. Each tree is trained on a random subset of the data and makes its own prediction. For any given input, such as whether a basket should be classified as fraudulent, each tree casts a "vote," and the final prediction is determined by a majority vote. Since the individual trees are trained on different subsets of the data and features, their errors tend to cancel each other out. This results in a more robust and accurate model than using a single decision tree (scikit-learn, 2025).

5.4.2 Why is this model good for finding fraud at self-checkout?

The RandomForestClassifier has several advantages for fraud detection. First, it effectively handles imbalanced datasets by allowing class weighting, which ensures that rare fraud cases are not overlooked. Second, it identifies complex, nonlinear patterns, such as the interplay between trust level, scan time, and number of voids, which are common in fraud scenarios. Third, since its predictions are based on an ensemble of trees, the model is less prone to overfitting and tends to generalize better than individual decision trees do. Fourth, it provides feature importance metrics that help staff understand which factors contribute to a fraud prediction. Finally, the algorithm is computationally efficient and can handle large tabular datasets, making it suitable for real-time monitoring at high-volume retail checkouts (scikit-learn, 2025).

5.4.3 How can we use the RandomForestClassifier in our project?

First, we prepare the data. Since most fraud cases occur within trust levels 1 and 2, we only retain baskets within these groups. Then, we engineer relevant features, such as the total number of items, the number of voids, scans without registration, and the total scan time. To address the class imbalance, since fraud cases are rare, we either apply the `class_weight` parameter with the value 'balanced' or manually assign higher importance to the minority class.

Next, we train a RandomForestClassifier using scikit-learn. We configure key hyperparameters, such as the number of trees and the maximum tree depth. The model is trained on 60% of the data using stratified sampling to preserve the class distribution. Then, we tune the hyperparameters using a validation set to improve recall, precision, and ROC-AUC, with a focus on fraud detection. After tuning, we examine the model's feature importance scores to understand which features influence its decisions. Finally, we evaluate the model's performance on the remaining 20% of the data and report accuracy, precision, recall, F1-score, and ROC-AUC to ensure that the model generalizes well to unseen baskets. If this model outperforms the alternatives, we will deploy it in the self-checkout system.

5.5 Generation of Test Design

As discussed in Section 4.3 of the paper, we found that balancing the dataset using SMOTE or under sampling the majority class decreased model performance. These techniques did not produce satisfactory results in our case. Therefore, we adopted a 60/20/20 data splitting strategy, allocating 60% of the data for training, 20% for validation, and 20% for testing. This approach provides a balanced framework for training, tuning, and evaluating the model. Additionally, we used a five-fold

cross-validation process during training to fine-tune the hyperparameters and find the optimal balance between bias and variance.

The training set (60%) is used to fit the model and identify patterns in the data. The validation set (20%) is used to fine-tune hyperparameters, compare different algorithms, and detect overfitting during model development. The test set (20%) is saved until the end of the process to evaluate the final model's performance on data it has never seen before. This provides an unbiased estimate of the model's generalization ability. The splitting process was implemented using the `train_test_split` function from the scikit-learn library, as shown below:

```
# Step 2: Proper train/val/test split (60% train, 20% val, 20% test)
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42
) # 0.25 x 0.8 = 0.2
```

Figure 20. Code executed in Jupyter Notebook for generating the test design

In the first split, 20% of the original data is set aside for the test set. The remaining 80% is temporarily stored in `X_(temp)` and `Y_(temp)`. In the second split, 25% of the temporary set is taken as validation data, corresponding to 20% of the original dataset ($0.25 \times 0.8 = 0.2$). This leaves 60% for training. Both splits use stratified sampling to maintain the same class distribution across the training, validation, and test sets. This is particularly important when dealing with imbalanced datasets because it ensures fair representation of each class in all subsets. This splitting strategy helps ensure that the model is well-trained, properly tuned, and fairly evaluated, supporting a more rigorous and reliable machine learning workflow.

5.6 Building the Model

Initially, we experimented with the models using their default parameters and observed significant overfitting. To address this issue, we applied 5-fold cross-validation and conducted hyperparameter tuning to identify the optimal settings for each model. Since false positives are five times more costly than false negatives in our context, we developed two separate strategies: (1) balanced cross-validation tuning, which focuses on standard class balancing and parameter optimization, and (2) cost-sensitive learning with threshold adjustment, which explicitly incorporates the cost difference by adjusting class weights and optimizing the decision threshold.

1. Balanced Cross-Validation Tuning

This strategy focuses on rigorous model selection and hyperparameter optimization to build an efficient, robust fraud detection model. First, we defined a set of evaluation metrics: ROC AUC, F1

score, precision, and recall. These metrics are essential in our scenario for capturing the various aspects of model performance, particularly given the significant class imbalance in our dataset. We used a five-fold stratified cross-validation strategy to ensure that each fold contained the same class distribution as the original dataset. This provided reliable and unbiased performance estimates during hyperparameter tuning. For each model, we set up hyperparameter grids with moderate values to prevent overfitting and enable generalization. For instance, in the random forest classifier, we only allowed a maximum depth of 6 because greater depths would lead to overfitting. Most importantly, we used class balancing techniques with measures such as `class_weight="balanced"` or `scale_pos_weight` to address the prevalence of both the majority and minority classes.

During training, each model went through an exhaustive grid search using the defined cross-validation strategy and metrics, with ROC AUC selected here as the primary metric for refitting and model selection. ROC AUC was chosen due to its strong discriminative ability. We recorded the best hyperparameters and corresponding validation scores for each metric. After tuning the models, we evaluated their validation performance and selected the model with the highest ROC AUC as the final candidate. We then assessed this final model on the hold-out test set to report its true generalization ability using all previously mentioned metrics (ROC-AUC, F1, precision, and recall). This rigorous, metric-driven approach aimed to produce a balanced and reliable fraud detection system with careful attention to class imbalance and model complexity.

2. Cost-sensitive learning with threshold adjustment

This strategy is designed to build a robust, cost sensitive fraud detection model that explicitly counts for the higher cost of false positive. The goal here would be achieve a higher precision than the one, achieved with strategy 1. Here, we have used the same evaluation metrics as in strategy 1. The objective with the metrics is not only to measure the overall accuracy but also to find a good balance between false positives and false negatives. Again, we have used a stratified cross validation to ensure that we maintain original class ratio in the data set, which is key in the cases of imbalanced classification problems. This would definitely help us to finetune the hyperparameters without biasing too much towards the majority class.

To address the cost asymmetry, we assign higher class weights to the minority (fraud) class in all classifiers (for example, `{0: 1, 1: 3}`), making the models more sensitive to fraud cases. For XGBoost, we adjust `scale_pos_weight` to further compensate for class imbalance. Our candidate models—Random Forest, XGBoost, Logistic Regression, and SVM—are each defined with their own grid of moderate hyperparameters.

For the Random Forest classifier, we systematically varied several key hyperparameters. These included the number of trees (`n_estimators`), which controls the ensemble size; the maximum depth

of each tree (`max_depth`), which limits model complexity to reduce overfitting; and the minimum number of samples required to split an internal node (`min_samples_split`) and the minimum number of samples required to be present at a leaf node (`min_samples_leaf`). The latter two help control tree granularity and generalisation. We also included the cost-complexity pruning parameter (`ccp_alpha`), which penalises overly complex trees to regulate overfitting. Additionally, we used bootstrap sampling (`bootstrap=True`) and a fixed proportion of data per tree (`max_samples`) to promote diversity and robustness within the forest, as well as the square root of the number of features at each split (`max_features='sqrt'`).

For XGBoost, we optimised the model across a range of hyperparameters that are specifically suited to imbalanced classification tasks. The `scale_pos_weight` parameter was tuned to emphasise the importance of the minority class, while the `subsample` and `colsample_bytree` parameters were adjusted to control for overfitting by introducing randomness in row and feature selection. The maximum tree depth (`max_depth`), learning rate (`learning_rate`), minimum child weight (`min_child_weight`) and gamma parameter (`gamma`), which defines the minimum loss reduction for additional partitioning, were all systematically explored. Regularisation was further managed through the L1 (`reg_alpha`) and L2 (`reg_lambda`) penalty terms.

For the logistic regression model embedded in a pipeline with feature standardisation, we explored different values for inverse regularisation strength (`C`), various penalty types (including L1 and L2) and the elastic net mixing parameter (`l1_ratio`), which determines the balance between L1 and L2 regularisation when using the 'saga' solver. The maximum number of iterations (`max_iter`) was set high enough to guarantee convergence, and class weighting was applied in the same way as for other models.

Within a standardised pipeline, the SVM model was also optimised for the regularisation parameter (`C`), which controls the trade-off between achieving low training and testing errors, as well as for the kernel type (`kernel`), kernel coefficient (`gamma`) and optimisation tolerance (`tol`). Probability estimation was enabled (`probability=True`) to allow for flexible threshold adjustment in subsequent steps.

Each of these models underwent hyperparameter tuning via an exhaustive grid search combined with stratified five-fold cross-validation. Multiple performance metrics were employed to ensure comprehensive and fair model evaluation, particularly in the presence of class imbalance: ROC AUC, F1 score, precision and recall. During the model selection process, the F1 score was prioritised for refitting in recognition of the need to balance precision and recall under cost-sensitive conditions. Following hyperparameter optimisation, an additional threshold adjustment procedure was performed, evaluating a range of probability thresholds (e.g. 0.5 to 0.9) on the validation set. This

step enabled us to select the classification threshold that yielded the highest F1 score, thereby further mitigating the impact of costly false positives. Overall, this rigorous, multi-stage optimisation framework ensured that each model was finely tuned for both statistical performance and alignment with the real-world business constraints of the fraud detection problem.

5.7 Assessing the Model

For assessing the models, we have used the following 4 metrics, which are ROC AUC, F1 score, precision and recall. ROC AUC (Receiver Operating Characteristic – Area Under the Curve) is a measure of a model's ability to discriminate between classes. It does this across all possible classification thresholds. In other words, it summarises the trade-off between true positive and false positive rates. A higher ROC AUC indicates a stronger ability to distinguish between fraudulent and non-fraudulent cases, regardless of the distribution of classes. Precision is key to quantifying the proportion of positive predictions that are actually correct, which is crucial in fraud detection, where the cost of false positives can be high. Recall, on the other hand, measures the proportion of actual fraud cases that are correctly identified by the model, reflecting the model's sensitivity to the minority class. The F1 score is the harmonic mean of precision and recall, providing a single metric that balances these two aspects. The F1 score is particularly appropriate in situations where both false positives and false negatives are unacceptable, as it captures both types of classification error in a single value. We make sure that our assessment reflects the model's overall discriminative power and its effectiveness in minimising costly errors and achieving balanced performance that aligns with the real-world requirements of fraud detection by taking ROC AUC, precision, recall, and F1 score into consideration. Having used two strategies in the model-building phase, we will present the evaluation metrics of both here and select one that best suits the business problem in the evaluation phase.

1. Balanced Cross-Validation Tuning

a. Random Forest Classifier

As shown in the table below, the Random Forest classifier shows strong overall performance, especially in distinguishing fraudulent from non-fraudulent baskets, as evidenced by its high ROC AUC of 0.9723. This indicates excellent discriminative power across all classification thresholds. With a recall score of 0.9547, the model effectively identifies over 95% of fraud cases. However, the relatively moderate precision score of 0.4778 means that more than half of the baskets flagged as fraudulent are actually not, which could lead to unnecessary interventions. The F1 score of 0.6369 strikes a reasonable balance between precision and recall, though it is affected by the lower

precision. Below, we have also attached the classification report and confusion matrix of random forest as well.

ROC AUC	F1	Precision	Recall
0.9723	0.6369	0.4778	0.9547

Table 2. Performance Score of Random Forest Classifier

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.83	0.90	28380
1	0.48	0.95	0.64	4745
accuracy			0.84	33125
macro avg	0.73	0.89	0.77	33125
weighted avg	0.92	0.84	0.86	33125

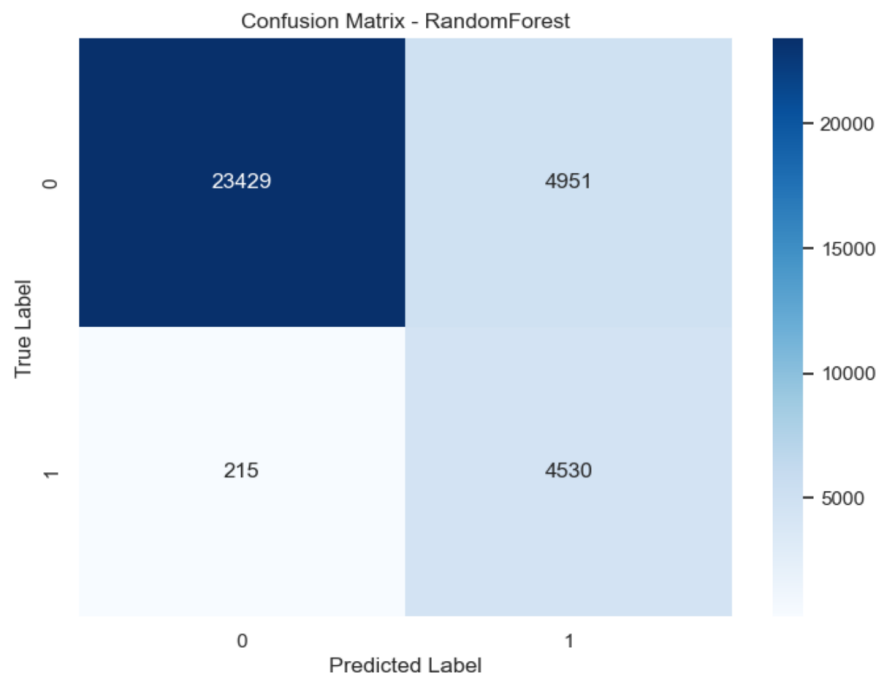


Figure 21. Confusion Matrix of the random forest classifier

b. XGBoost Classifier

The table below shows that the XGBoost classifier model has a ROC AUC of 0.9979, which indicates an exceptional ability to distinguish between fraudulent and non-fraudulent transactions, with the model performing this task almost perfectly regardless of the chosen decision threshold. An F1 score of 0.8425 indicates a favourable balance between precision and recall, suggesting that the model strikes a good balance between correctly identifying fraudulent transactions and minimising false alarms. With a precision value of 0.7284, approximately 73% of transactions flagged as fraudulent

by the model are true frauds. Meanwhile, the recall of 0.9989 indicates that the model is extremely sensitive to fraud cases, successfully capturing nearly all actual fraudulent transactions and thus minimizing false negatives. We have attached the classification report and confusion matrix as well.

ROC AUC	F1	Precision	Recall
0.9979	0.8425	0.7284	0.9989

Table 3. Performance score of XGBoost Classifier

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.94	0.97	28380
1	0.73	1.00	0.84	4745
accuracy			0.95	33125
macro avg	0.86	0.97	0.91	33125
weighted avg	0.96	0.95	0.95	33125

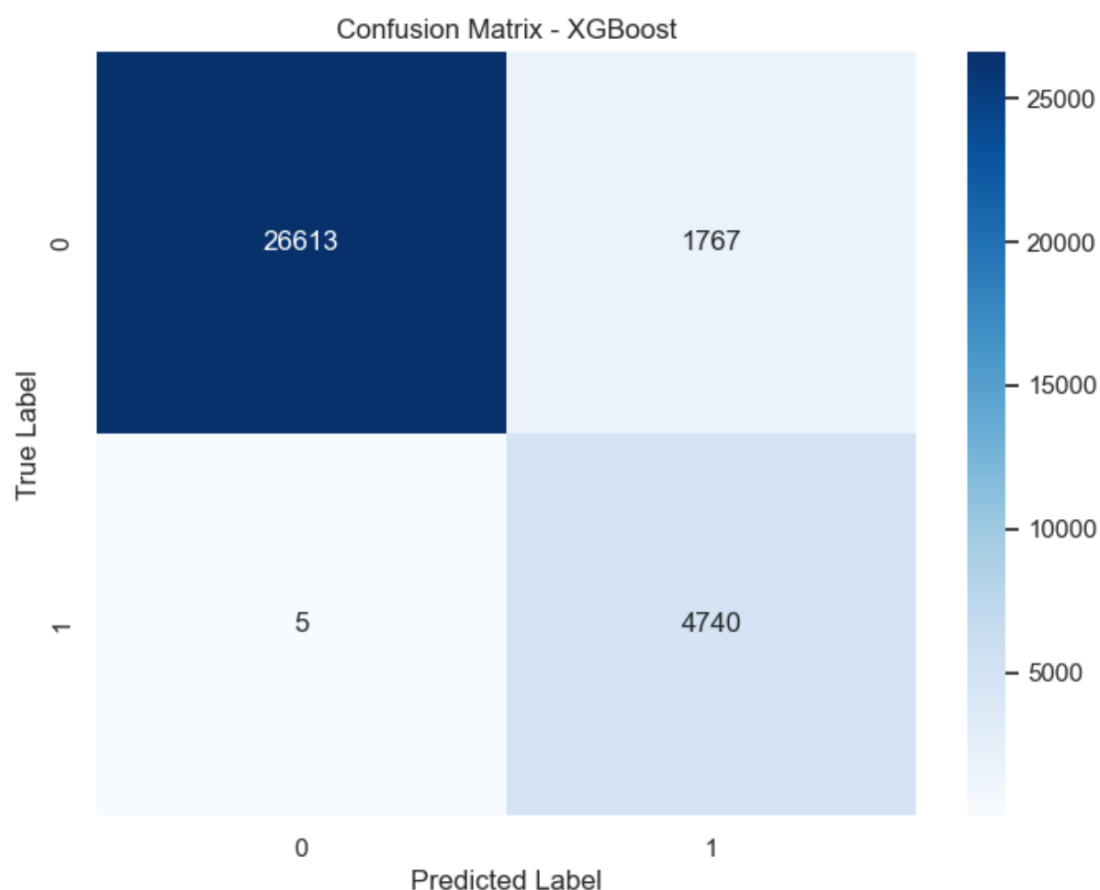


Figure 22. Confusion Matrix of XGBoost classifier

c. SVM

The SVM's ROC AUC of 0.9968 indicates that the model is highly effective at distinguishing between fraudulent and non-fraudulent cases, achieving near-perfect classification across various thresholds. An F1 score of 0.8565 reveals a favourable balance between precision and recall, reflecting the model's ability to identify fraud cases while minimising false positives and false negatives. With a precision score of 0.7524, the SVM correctly classifies around 75% of flagged fraud cases as genuine fraud. Furthermore, a recall of 0.9941 shows that the SVM successfully detects almost all actual fraud cases, thereby reducing the risk of fraudulent transactions being overlooked. The classification report and confusion matrix has been provided as well below.

ROC AUC	F1	Precision	Recall
0.9968	0.8565	0.7524	0.9941

Table 4. Performance score of SVM

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.95	0.97	28380
1	0.75	0.99	0.86	4745
accuracy			0.95	33125
macro avg	0.88	0.97	0.91	33125
weighted avg	0.96	0.95	0.95	33125

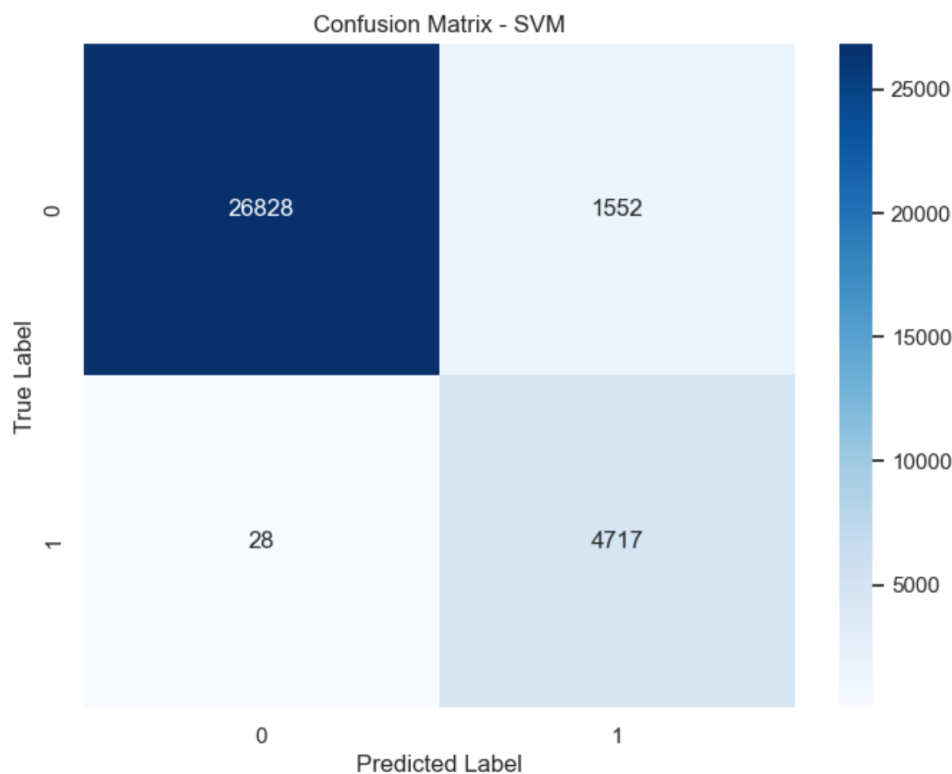


Figure 23. Confusion Matrix of SVM

d. Logistic Regression

With a ROC AUC score of 0.9960, the logistic regression model demonstrates its ability to distinguish between fraudulent and non-fraudulent transactions across various thresholds, showcasing near-optimal classification performance. The F1 score of 0.8769 indicates an excellent balance between precision and recall, reflecting the model’s ability to detect fraud while minimising false positives and false negatives. A precision score of 0.7909 indicates that almost 80% of transactions identified as fraudulent are true positives. Furthermore, a recall of 0.9838 indicates the model’s high sensitivity, as it successfully detects the majority of actual fraud cases. The classification report and confusion matrix has been given below.

ROC AUC	F1	Precision	Recall
0.9960	0.8769	0.7909	0.9838

Table 5. Performance score of Logistic Regression

Classification Report:					
	precision	recall	f1-score	support	
0	1.00	0.96	0.98	28380	
1	0.79	0.98	0.88	4745	
accuracy			0.96	33125	
macro avg	0.89	0.97	0.93	33125	
weighted avg	0.97	0.96	0.96	33125	

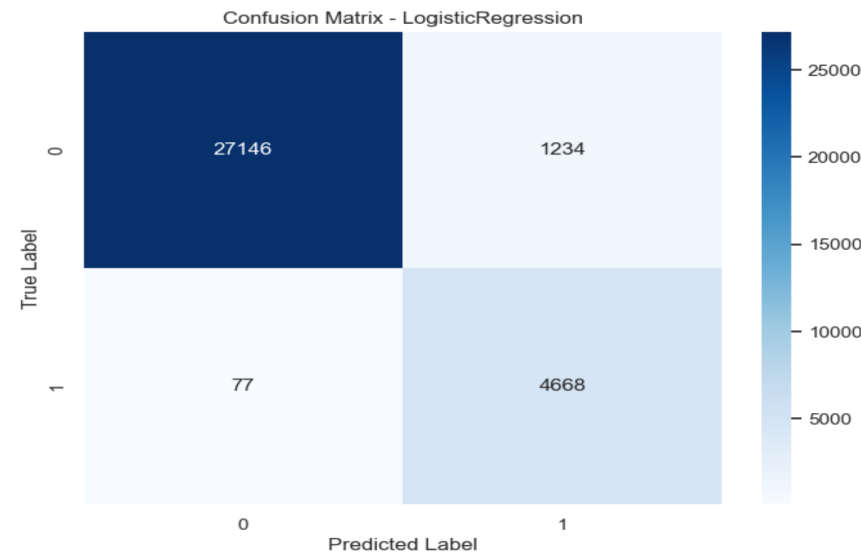


Figure 24. Confusion Matrix of Logistic Regression

As previously mentioned, the final model is selected based on the ROC-AUC score. Therefore, the final model is XGBoost, with a ROC-AUC score of 0.9979. On the test set, this model achieved the following score:

ROC AUC	F1	Precision	Recall
0.9978	0.8483	0.7304	0.9977

Table 6. Performance score of the final model, XGBoost classifier

2. Cost-sensitive learning with threshold adjustment

a. Random Forest Classifier

The Random Forest classifier performs moderately well across the suite of evaluation metrics. With an ROC AUC score of 0.9578, it demonstrates a strong ability to differentiate between fraudulent and non-fraudulent transactions across a range of thresholds. The F1 score of 0.6986 indicates a moderate balance between precision and recall. This suggests that, although the model can identify a significant proportion of fraud cases, there is scope to reduce classification errors further. With a precision of 0.5744, the model correctly classifies approximately 57% of flagged transactions as actual fraud, which is higher than the random forest model in strategy 1. The recall of 0.8913 shows that the model can detect almost 90% of actual fraud cases, demonstrating reasonable sensitivity to fraudulent activity. Overall, Strategy 2 seems to be more balanced and well generalised.

ROC AUC	F1	Precision	Recall
0.9578	0.6986	0.5744	0.8913

Table 7. Performance score of random forest classifier in strategy 2

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.89	0.93	28380
1	0.57	0.89	0.70	4745
accuracy			0.89	33125
macro avg	0.78	0.89	0.82	33125
weighted avg	0.92	0.89	0.90	33125

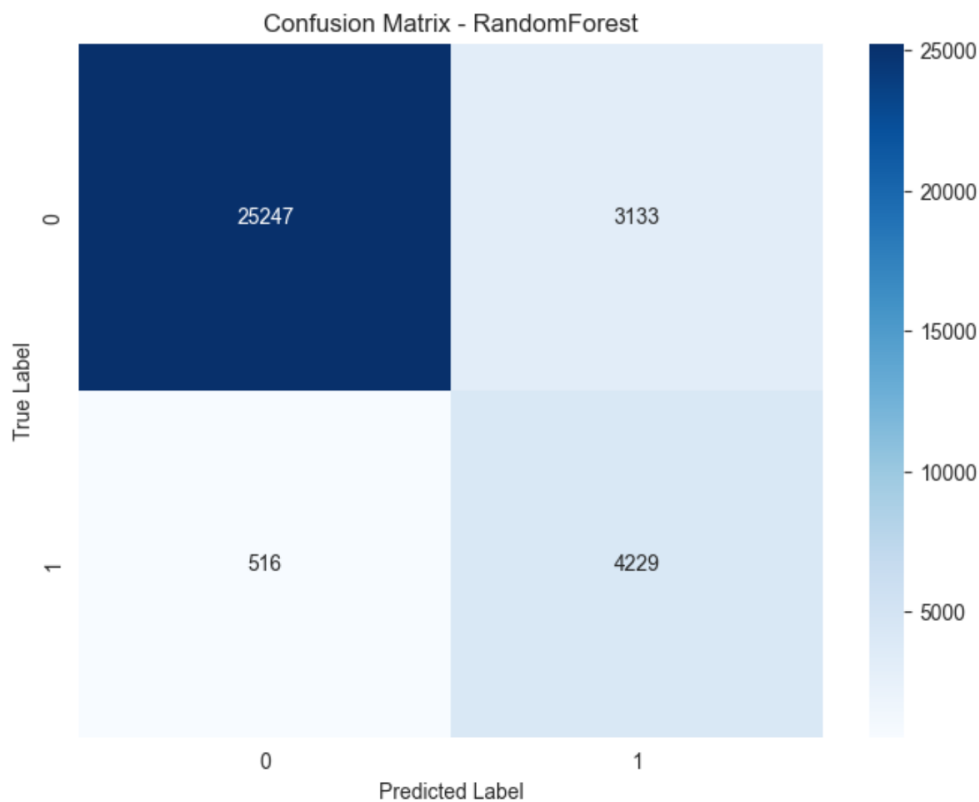


Figure 25. Confusion Matrix of Random Forest in strategy 2

b. XGBoost Classifier

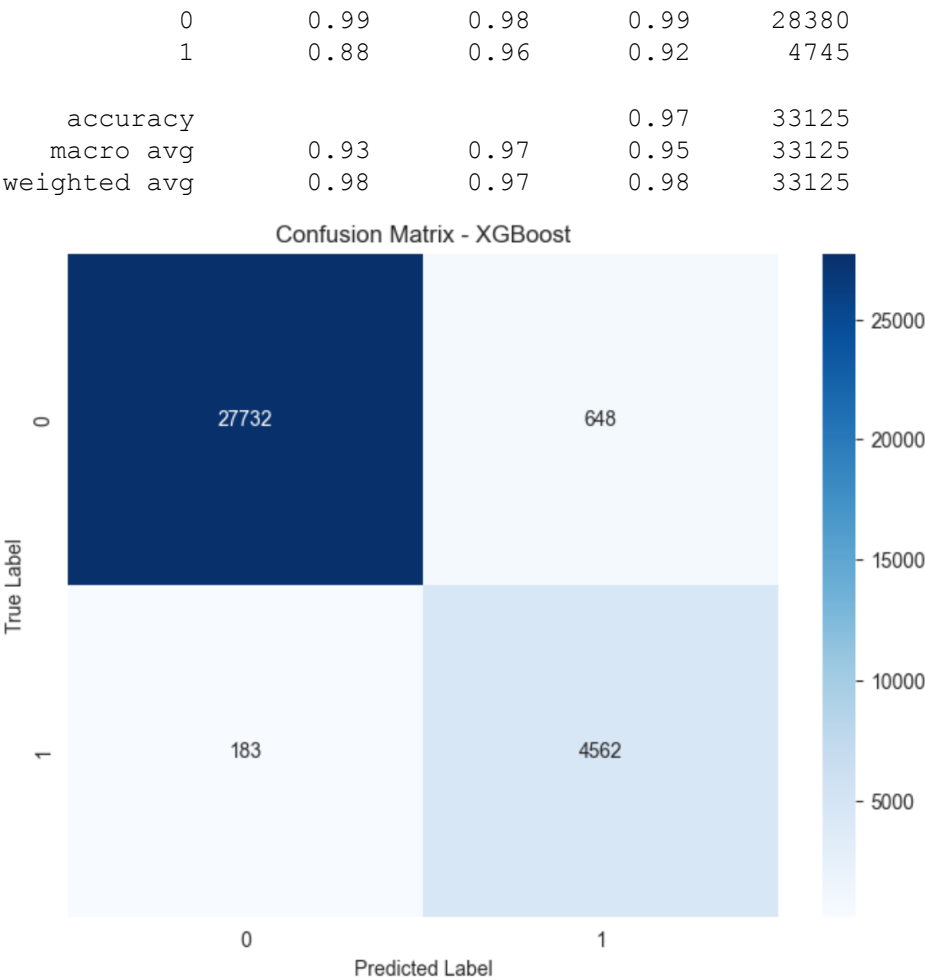
The XGBoost classifier demonstrates exceptional performance across all key metrics under the cost-sensitive learning with threshold adjustment strategy (strategy 2). With an ROC AUC of 0.9961, the model has an excellent ability to distinguish between fraudulent and non-fraudulent transactions across a range of classification thresholds. The high F1 score of 0.9165 reflects an excellent balance between precision and recall, indicating that the model is highly effective in identifying fraud cases while minimising both types of classification error. The precision value of 0.8756 shows that almost 88% of transactions flagged as fraudulent are actually fraudulent, which is important in a cost-sensitive context where false positives incur significant expense. Furthermore, a recall value of 0.9614 shows that the model successfully identifies over 96% of all actual fraud cases, ensuring that very few fraudulent transactions go undetected.

ROC AUC	F1	Precision	Recall
0.9961	0.9165	0.8756	0.9614

Table 8. Performance score of XGBoost classifier in strategy 2

Classification Report:

precision	recall	f1-score	support
-----------	--------	----------	---------



Classification Report:

	precision	recall	f1-score	support
0	1.00	0.96	0.98	28380
1	0.82	0.98	0.89	4745
accuracy			0.97	33125
macro avg	0.91	0.97	0.93	33125
weighted avg	0.97	0.97	0.97	33125

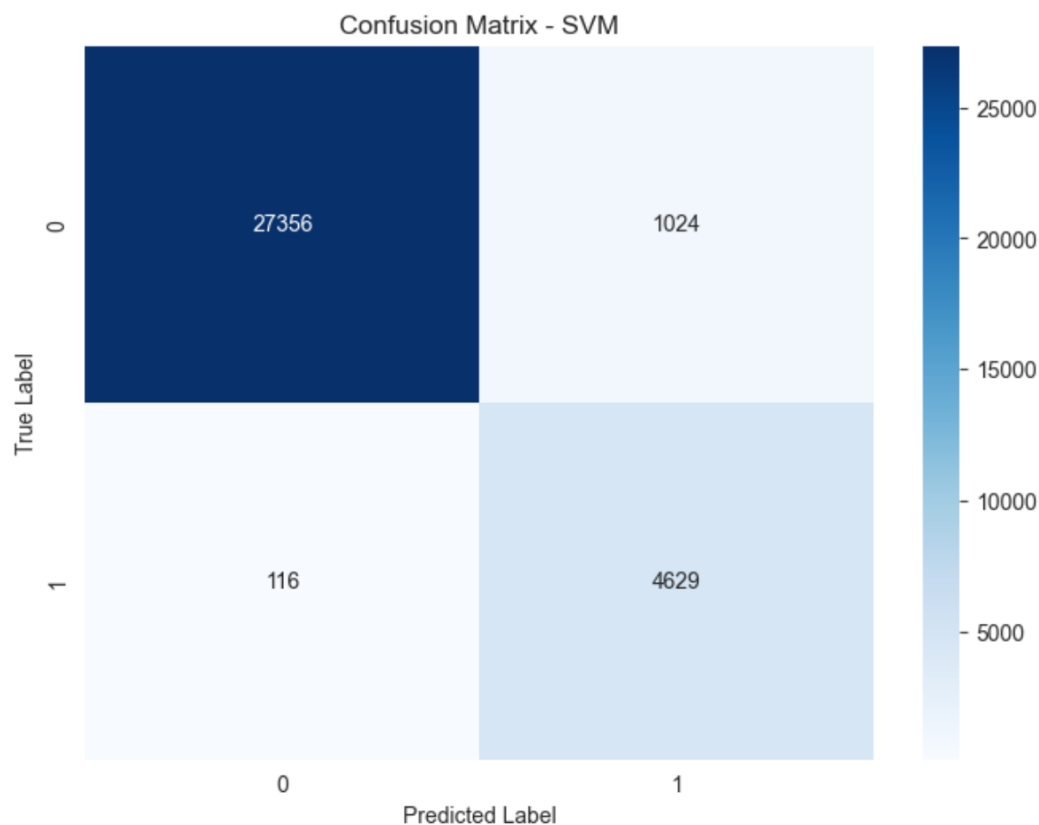


Figure 27. Confusion Matrix of SVM in strategy 2

d. Logistic Regression

With an ROC AUC of 0.9959, the logistic regression model demonstrates excellent discriminative capability, almost perfectly distinguishing between fraudulent and non-fraudulent cases across various decision thresholds. The F1 score of 0.8999 reflects a high degree of balance between precision and recall, highlighting the model's effectiveness in identifying fraud cases while minimising false positives and false negatives. A precision score of 0.8401 indicates that around 84% of transactions flagged as fraudulent by the model are genuine, which is vital for minimising the costs of unnecessary investigations. A recall of 0.9688 shows that the model successfully identifies over

96% of actual fraud cases, maintaining high sensitivity and ensuring that most fraudulent activity is detected.

ROC AUC	F1	Precision	Recall
0.9959	0.8999	0.8401	0.9688

Table 10. Performance score of Logistic Regression in strategy 2

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.97	0.98	28380
1	0.84	0.97	0.90	4745
accuracy			0.97	33125
macro avg	0.92	0.97	0.94	33125
weighted avg	0.97	0.97	0.97	33125

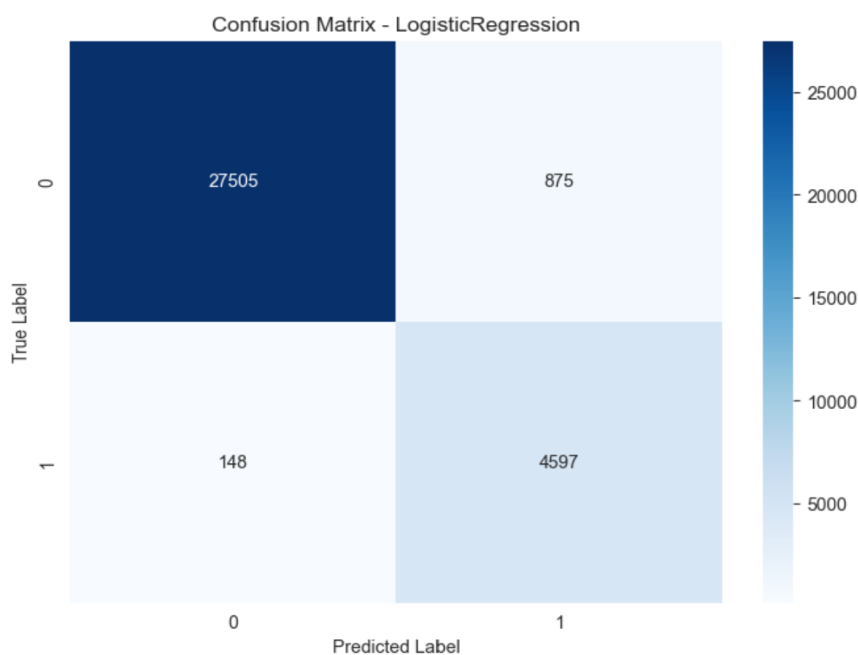


Figure 28. Confusion Matrix of Logistic Regression in strategy 2

As previously mentioned, the final model is selected based on the F1 score. Therefore, the final model is XGBoost, as it achieved the best F1 score of 0.9165. The optimal threshold was also found to be 0.5. On the test set, this model achieved the following score:

ROC AUC	F1	Precision	Recall
0.9961	0.9189	0.8781	0.9638

Table 11. Performance score of the final model, XGBoost in strategy 2

6 Evaluation

The evaluation phase comprised three key steps. First, the model's results were assessed against the predefined business success criteria. Second, a comprehensive review of all modelling and validation procedures was conducted. Third, the final actions were determined, including whether the model should be deployed in a production environment. As previously outlined, the main business requirement was that the model should achieve a minimum of 80% in both precision and recall, with a particular focus on minimising the cost of false positives given their substantially greater business impact compared to false negatives. In light of these criteria, cost-sensitive learning with threshold adjustment (strategy 2) was identified as the optimal solution as it achieved a superior balance between precision and recall and consistently exceeded the required thresholds, thereby aligning closely with the business's operational objectives.

Consequently, the XGBoost classifier utilising Strategy 2 was selected as the final model for deployment. This classifier attained an F1 score of 0.9165 during validation. To maximise its generalisability and leverage the entire available dataset, the model was retrained using the combined training and validation sets. Subsequent testing on an independent test set confirmed the model's robust performance, with the model achieving an ROC AUC of 0.9959, an F1 score of 0.9135, a precision of 0.8701 and a recall of 0.9614, as shown in Table 12. These results not only satisfy, but exceed, the initial project success criteria. Given the model's strong performance across all relevant evaluation metrics, the final XGBoost model was saved in a serialized format using Joblib for future deployment and inference tasks.

ROC AUC	F1	Precision	Recall
0.9959	0.9135	0.8701	0.9614

Table 12. Performance score of the final model

7 Findings

This analytical project was a valuable and formative experience, as it was our first opportunity to carry out a project that adhered fully to the CRISP-DM framework. Progressing through all six phases of the framework in a structured manner was instrumental in guiding our work and ensuring we achieved our project objectives. Applying theoretical knowledge in a practical context presented us with significant learning opportunities, especially with regard to adapting to new tools such as Jupyter Notebook and Orange, both of which were crucial for carrying out our analysis. The size and complexity of the dataset, consisting of around half a million records, mirrored real-world scenarios and challenged us to develop new competencies in handling large volumes of data. Throughout the project, we deepened our understanding of various classification algorithms and their practical applications.

The coaching sessions with Prof. Andreas Reber were particularly helpful, providing expert guidance, timely feedback and adjustments that greatly contributed to our success. As a team, we are very satisfied with the results achieved and grateful for the hands-on analytical experience gained through this project. Upon reflection, one area for improvement in future projects is the data preparation phase. Our experience has emphasised the critical importance of dedicating substantial time and effort to data cleaning and pre-processing, as this phase lays the groundwork for the development of effective and reliable models. Going forward, allocating additional resources and time to this stage will further improve project outcomes and analytical integrity.

References

- scikit-learn. (n.d.). *sklearn.linear_model.LogisticRegression*. Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- XGBoost Documentation. (n.d.). *XGBoost Documentation (Release 3.0.0)*. Retrieved from https://xgboost.readthedocs.io/en/release_3.0.0/
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. Retrieved from <https://scikit-learn.org/stable/modules/svm.html>
- Data Science PM. (2024). *What is CRISP DM?*. Retrieved from <https://www.datascience-pm.com/crisp-dm-2/>
- Imbalanced learn. (2025). *SMOTE*. Retrieved from https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html
- Orange Data Mining. (2025). *Orange*. Retrieved from <https://orangedatamining.com/>
- Pandas. (2024). *Pandas*. Retrieved from <https://pandas.pydata.org/>
- scikit-learn. (2025). *RandomForestClassifier*. Retrieved from <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

List of aids

Aid	Usage	Affected areas
DeepL Write	Optimization of individual text passages	Entire work
Orange	Data Analysis	Chapter 3.2
Mendeley	Preparation of in-text citations and reference list	In-text citations and Reference list
ChatGPT	Project brainstorming and code generation	Chapter 2,3,3,5 and 6 and Jupyter Notebook
GitHub Copilot	Code generation	Jupyter Notebook

List of Figures

Figure 1. CRISP-DM Model and its 6 Phases	4
<i>Figure 2. Almost all fraud occurs at trust level 1, with only a small amount at level 2. Baskets from levels 3 through 6 are all free of fraud.....</i>	<i>8</i>
<i>Figure 3. Comparison of Grand Total Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>8</i>
<i>Figure 4. Comparison of Quantity Modifications Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>9</i>
<i>Figure 5. Comparison of Line Item Voids Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>10</i>
<i>Figure 6. Comparison of Scans Without Registration Distribution between Fraudulent and Non-Fraudulent cases.....</i>	<i>11</i>
<i>Figure 7. Comparison of Total Scan Time Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>12</i>
<i>Figure 8. Comparison of Scanned Line Items Per Second Distribution between Fraudulent and Non-Fraudulent cases.....</i>	<i>13</i>
<i>Figure 9. Comparison of Value Per Second Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>14</i>
<i>Figure 10. Comparison of Line-Item Voids Per Position Distribution between Fraudulent and Non-Fraudulent cases.....</i>	<i>15</i>
<i>Figure 11. Correlation between variables</i>	<i>16</i>
<i>Figure 12. Violin plot of the relationship between totalScanTimeInSeconds and the fraud variable</i>	<i>16</i>
<i>Figure 13. Violin plot of the relationship between trustLevel and the fraud variable</i>	<i>17</i>
<i>Figure 14. Code snippet executed in Jupyter Notebook checking for null values.....</i>	<i>17</i>
<i>Figure 15. Comparison of Total Items Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>19</i>
<i>Figure 16. Comparison of Average Value Distribution between Fraudulent and Non-Fraudulent cases</i>	<i>19</i>
<i>Figure 17. Code snippet executed in Jupyter Notebook to remove highly correlated features.....</i>	<i>21</i>
<i>Figure 18. Code snippet executed in Jupyter Notebook to split data</i>	<i>21</i>
	49

<i>Figure 19. Combined feature importance scores from all models.....</i>	<i>24</i>
<i>Figure 20. Code executed in Jupyter Notebook for generating the test design</i>	<i>31</i>
<i>Figure 21. Confusion Matrix of the random forest classifier.....</i>	<i>35</i>
<i>Figure 22. Confusion Matrix of XGBoost classifier.....</i>	<i>36</i>
<i>Figure 23. Confusion Matrix of SVM</i>	<i>38</i>
<i>Figure 24. Confusion Matrix of Logistic Regression.....</i>	<i>39</i>
<i>Figure 25. Confusion Matrix of Random Forest in strategy 2.....</i>	<i>40</i>
<i>Figure 26. Confusion Matrix of XGBoost in strategy 2</i>	<i>41</i>
<i>Figure 27. Confusion Matrix of SVM in strategy 2.....</i>	<i>42</i>
<i>Figure 28. Confusion Matrix of Logistic Regression in strategy 2</i>	<i>43</i>

List of Tables

<i>Table 1.</i> Data Description	7
Table 2. Performance Score of Random Forest Classifier.....	35
<i>Table 3.</i> Performance score of XGBoost Classifier.....	36
<i>Table 4.</i> Performance score of SVM	37
<i>Table 5.</i> Performance score of Logistic Regression	38
<i>Table 6.</i> Performace score of the final model, XGBoost classifier	39
<i>Table 7.</i> Performance score of random forest classifier in strategy 2	39
<i>Table 8.</i> Performance score of XGBoost classifier in strategy 2	40
<i>Table 9.</i> Performance score of SVM in strategy 2	41
<i>Table 10.</i> Performance score of Logistic Regression in strategy 2	43
Table 11. Performance score of the final model, XGBoost in strategy 2	43
<i>Table 12.</i> Performance score of the final model	45

List of Abbreviations

AUC	Area Under Curve
CRISP-DM	Cross Industry Standard Process for Data Mining
ROC	Receiver-Operating characteristic curve
NaN	Not a Number
NA	Not Applicable
RBF	Radial Basis Function
SMOTE	Synthetic Minority Over-sampling Technique
SVM	Support Vector Machine
XGBoost	eXtreme Gradient Boosting

Appendix A

a. Additional Material

Material	Accessibility
GitHub Repository, where the entire project analysis is conducted.	GitHub Link